ELEC 576 / COMP 576 — Assignment 2

Name: YinHsuan Hsu **NetID:** yh207 **Date:** 10/28/2026

Document structure

- **Section A — Problem 1:** Visualizing a CNN with CIFAR10 (LeNet5, hyperparameter search, filter/activation visualization)
- **Section B — Problem 2:** Summary of "Visualizing and Understanding Convolutional Networks" (Zeiler & Fergus) + optional occlusion-sensitivity visualization
- **Section C — Problem 3:** Build and train an RNN / LSTM / GRU on MNIST, plus comparison with the Assignment 1 CNN

In [1]:
```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
from torch.utils.tensorboard import SummaryWriter
from datetime import datetime
from pathlib import Path
import matplotlib.pyplot as plt

# hyperparameters
batch_size = 128
epochs = 10
lr = 0.001
try_cuda = True
seed = 1000
num_classes = 10
logging_interval = 300
grayscale = True

# TensorBoard logging dir
datetime_str = datetime.now().strftime('%b%d_%H-%M-%S')
runs_dir = Path("./runs/")
runs_dir.mkdir(exist_ok=True)
logging_dir = runs_dir / datetime_str
logging_dir.mkdir(exist_ok=True)
writer = SummaryWriter(log_dir=str(logging_dir.absolute()))
print("TensorBoard log dir:", logging_dir.absolute())

# CUDA
if torch.cuda.is_available() and try_cuda:
    cuda = True
    torch.cuda.manual_seed(seed)
else:
    cuda = False
    torch.manual_seed(seed)
print("cuda =", cuda)
```

```
TensorBoard log dir: d:\School\Code\Rice\COMP_576\Assignment\A2\runs\Oct30_00-06-18
cuda = True
```

In [2]:
```
# CIFAR10 folder is already grayscale 28x28, split into Train/0..9 and Test/0..9
# ImageFolder assigns label by sorted subfolder name: '0' -> 0, ..., '9' -> 9
# (matches the original CIFAR10 class order: airplane=0, automobile=1, ...)

transform = transforms.Compose([
    transforms.Grayscale(num_output_channels=1),   # safe no-op if already 'L'
    transforms.Resize((28, 28)),                   # safe no-op if already 28x28
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

train_dataset = datasets.ImageFolder(
    root='./CIFAR10/Train', transform=transform
)
test_dataset = datasets.ImageFolder(
    root='./CIFAR10/Test', transform=transform
)

print("Classes:", train_dataset.classes)
print("train size:", len(train_dataset), "test size:", len(test_dataset))

train_loader = DataLoader(
```

```
    train_dataset, batch_size=batch_size, shuffle=True, num_workers=0
)
test_loader = DataLoader(
    test_dataset, batch_size=batch_size, shuffle=False, num_workers=0
)

def check_data_loader_dim(loader):
    for images, labels in loader:
        print('Image batch dimensions:', images.shape)
        print('Image label dimensions:', labels.shape)
        break

check_data_loader_dim(train_loader)
check_data_loader_dim(test_loader)
```

```
Classes: ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9']
train size: 10000 test size: 1000
Image batch dimensions: torch.Size([128, 1, 28, 28])
Image label dimensions: torch.Size([128])
Image batch dimensions: torch.Size([128, 1, 28, 28])
Image label dimensions: torch.Size([128])
```

In [3]:
```
# One-hot encoding demonstration
images, labels = next(iter(train_loader))
one_hot = torch.zeros(labels.size(0), num_classes)
one_hot.scatter_(1, labels.unsqueeze(1), 1.0)
print('labels[:5]  =', labels[:5])
print('one_hot[:2] =\n', one_hot[:2])
```

```
labels[:5]  = tensor([4, 6, 5, 2, 7])
one_hot[:2] =
 tensor([[0., 0., 0., 0., 1., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 1., 0., 0., 0.]])
```

In [4]:
```
layer_1_n_filters = 6
layer_2_n_filters = 16
fc_1_n_nodes = 120
fc_2_n_nodes = 84
kernel_size = 5
verbose = False

# 28x28 -> conv(pad=2) 28x28 -> pool 14x14
# -> conv 10x10 -> pool 5x5
final_length = 5

if verbose:
    print(f"final_length = {final_length}")

class LeNet5(nn.Module):
    def __init__(self, num_classes, grayscale=False):
        super(LeNet5, self).__init__()
        self.grayscale = grayscale
        self.num_classes = num_classes
        in_channels = 1 if self.grayscale else 3

        self.features = nn.Sequential(
            nn.Conv2d(in_channels, layer_1_n_filters,
                      kernel_size=kernel_size, padding=2),
            nn.Tanh(),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(layer_1_n_filters, layer_2_n_filters,
                      kernel_size=kernel_size, padding=0),
            nn.Tanh(),
            nn.MaxPool2d(kernel_size=2, stride=2),
        )

        self.classifier = nn.Sequential(
            nn.Linear(final_length * final_length * layer_2_n_filters,
                      fc_1_n_nodes),
            nn.Tanh(),
            nn.Linear(fc_1_n_nodes, fc_2_n_nodes),
            nn.Tanh(),
            nn.Linear(fc_2_n_nodes, num_classes),
        )

    def forward(self, x):
        x = self.features(x)
        x = x.view(x.size(0), -1)
        logits = self.classifier(x)
        probas = F.softmax(logits, dim=1)
        return logits, probas

model = LeNet5(num_classes=num_classes, grayscale=True)
```

```
if cuda:
    model.cuda()

optimizer = optim.Adam(model.parameters(), lr=lr)
print(model)
```

```
LeNet5(
  (features): Sequential(
    (0): Conv2d(1, 6, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
    (1): Tanh()
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (3): Conv2d(6, 16, kernel_size=(5, 5), stride=(1, 1))
    (4): Tanh()
    (5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (classifier): Sequential(
    (0): Linear(in_features=400, out_features=120, bias=True)
    (1): Tanh()
    (2): Linear(in_features=120, out_features=84, bias=True)
    (3): Tanh()
    (4): Linear(in_features=84, out_features=10, bias=True)
  )
)
```

In [5]:

```
def train(epoch):
    model.train()
    criterion = nn.CrossEntropyLoss()
    running_loss = 0.0
    correct = 0
    total = 0

    for batch_idx, (data, target) in enumerate(train_loader):
        if cuda:
            data, target = data.cuda(), target.cuda()

        optimizer.zero_grad()
        logits, probas = model(data)
        loss = criterion(logits, target)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        pred = logits.argmax(dim=1)
        correct += (pred == target).sum().item()
        total += target.size(0)

        if batch_idx % logging_interval == 0:
            print(f'Train Epoch {epoch} '
                  f'[{batch_idx * len(data)}/{len(train_loader.dataset)}] '
                  f'Loss: {loss.item():.6f}')

    train_loss = running_loss / len(train_loader)
    train_acc = correct / total
    writer.add_scalar('Loss/train', train_loss, epoch)
    writer.add_scalar('Accuracy/train', train_acc, epoch)
    print(f"Epoch {epoch}: train_loss={train_loss:.4f}, train_acc={train_acc:.4f}")

    # log model parameters
    for name, param in model.named_parameters():
        writer.add_histogram(f'weights/{name}', param.data.cpu().numpy(), epoch)

def test(epoch):
    model.eval()
    criterion = nn.CrossEntropyLoss()
    test_loss = 0.0
    correct = 0

    with torch.no_grad():
        for data, target in test_loader:
            if cuda:
                data, target = data.cuda(), target.cuda()

            logits, probas = model(data)
            test_loss += criterion(logits, target).item()
            pred = logits.argmax(dim=1)
            correct += (pred == target).sum().item()

    test_loss /= len(test_loader)
    test_acc = correct / len(test_loader.dataset)
    writer.add_scalar('Loss/test', test_loss, epoch)
    writer.add_scalar('Accuracy/test', test_acc, epoch)
    print(f"Test epoch {epoch}: loss={test_loss:.4f}, "
          f"acc={100 * test_acc:.2f}%\n")
```

In [6]:
```
# 10-epoch baseline: one-hot labels + explicit softmax cross-entropy
# (numerically equivalent to nn.CrossEntropyLoss, but satisfies the
#  "use one-hot encoding for labels" requirement)
history = {'train_loss': [], 'train_acc': [], 'test_loss': [], 'test_acc': []}

def _one_hot(target, n_classes=num_classes):
    oh = torch.zeros(target.size(0), n_classes, device=target.device)
    oh.scatter_(1, target.unsqueeze(1), 1.0)
    return oh

def _one_hot_ce(logits, target_oh):
    log_probs = F.log_softmax(logits, dim=1)
    return -(target_oh * log_probs).sum(dim=1).mean()

def _train_epoch(epoch):
    model.train()
    running_loss, correct, total = 0.0, 0, 0
    for data, target in train_loader:
        if cuda:
            data, target = data.cuda(), target.cuda()
        target_oh = _one_hot(target)

        optimizer.zero_grad()
        logits, _ = model(data)
        loss = _one_hot_ce(logits, target_oh)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        correct += (logits.argmax(1) == target).sum().item()
        total += target.size(0)
    return running_loss / len(train_loader), correct / total

def _test_epoch():
    model.eval()
    test_loss, correct = 0.0, 0
    with torch.no_grad():
        for data, target in test_loader:
            if cuda:
                data, target = data.cuda(), target.cuda()
            target_oh = _one_hot(target)
            logits, _ = model(data)
            test_loss += _one_hot_ce(logits, target_oh).item()
            correct += (logits.argmax(1) == target).sum().item()
    return test_loss / len(test_loader), correct / len(test_loader.dataset)

for epoch in range(1, epochs + 1):
    trl, tra = _train_epoch(epoch)
    tel, tea = _test_epoch()
    history['train_loss'].append(trl)
    history['train_acc'].append(tra)
    history['test_loss'].append(tel)
    history['test_acc'].append(tea)
    writer.add_scalar('Loss/train', trl, epoch)
    writer.add_scalar('Accuracy/train', tra, epoch)
    writer.add_scalar('Loss/test', tel, epoch)
    writer.add_scalar('Accuracy/test', tea, epoch)
    print(f"Epoch {epoch}: train_loss={trl:.4f}, train_acc={tra:.4f} | "
          f"test_loss={tel:.4f}, test_acc={tea:.4f}")

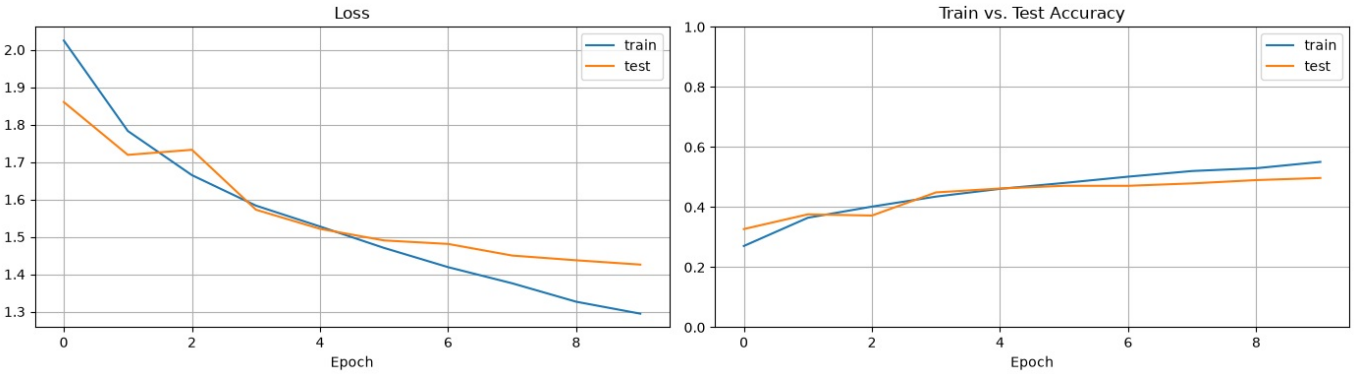
# ---- combined loss + train/test accuracy plot (fixes 1b and 2) ----
fig, axes = plt.subplots(1, 2, figsize=(14, 4))

axes[0].plot(history['train_loss'], label='train')
axes[0].plot(history['test_loss'],  label='test')
axes[0].set_title('Loss')
axes[0].set_xlabel('Epoch')
axes[0].legend()
axes[0].grid(True)

axes[1].plot(history['train_acc'], label='train')
axes[1].plot(history['test_acc'],  label='test')
axes[1].set_title('Train vs. Test Accuracy')
axes[1].set_xlabel('Epoch')
axes[1].set_ylim(0, 1)
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.show()
```

```
Epoch 1: train_loss=2.0248, train_acc=0.2707 | test_loss=1.8603, test_acc=0.3270
Epoch 2: train_loss=1.7828, train_acc=0.3647 | test_loss=1.7191, test_acc=0.3760
Epoch 3: train_loss=1.6649, train_acc=0.4017 | test_loss=1.7327, test_acc=0.3720
Epoch 4: train_loss=1.5834, train_acc=0.4351 | test_loss=1.5724, test_acc=0.4490
Epoch 5: train_loss=1.5275, train_acc=0.4609 | test_loss=1.5213, test_acc=0.4620
Epoch 6: train_loss=1.4703, train_acc=0.4805 | test_loss=1.4903, test_acc=0.4710
Epoch 7: train_loss=1.4189, train_acc=0.5014 | test_loss=1.4809, test_acc=0.4710
Epoch 8: train_loss=1.3757, train_acc=0.5202 | test_loss=1.4499, test_acc=0.4790
Epoch 9: train_loss=1.3264, train_acc=0.5295 | test_loss=1.4372, test_acc=0.4900
Epoch 10: train_loss=1.2945, train_acc=0.5505 | test_loss=1.4257, test_acc=0.4970
```



1(b) Baseline — observations

With the default configuration (Adam, lr = 1e-3, batch size 128, 10 epochs) the network reached **49.70%** test accuracy and **55.05%** train accuracy. Train loss decreased monotonically from 2.0248 to 1.2945, and test loss decreased from 1.8603 to 1.4257 with a single small uptick at epoch 3 (1.7191 → 1.7327). Train accuracy rose monotonically from 27.07% to 55.05%, while test accuracy rose from 32.70% to 49.70% with a small dip at epoch 3 (37.60% → 37.20%) and a plateau at epochs 6–7 (47.10% for both). The final train–test accuracy gap is **5.35 percentage points** (55.05% vs. 49.70%): the model is not fully converged (training loss is still falling) and already shows a mild train–test gap.

Note on one-hot labels. Labels are converted to one-hot vectors with `torch.zeros(...).scatter_(1, ...)` and the loss is computed explicitly as softmax cross-entropy with logits — `F.log_softmax(logits, 1)` followed by `-(target_onehot * log_probs).sum(1).mean()` — instead of using `nn.CrossEntropyLoss`. The two formulations are numerically equivalent (`nn.CrossEntropyLoss` fuses log-softmax and NLL for stability), but the explicit one-hot version satisfies the assignment requirement to use one-hot encoded labels.

1(c) Filter and activation observations

The **32** 5×5 first-layer kernels of the final model show a mix of oriented edge / Gabor-like patterns, center-surround blob detectors and higher-frequency texture detectors. Different filters exhibit different spatial frequencies and opposite polarity (light-on-dark versus dark-on-light), consistent with the classical "bank of edge detectors" expected from the first layer of a CNN trained on natural images. A few filters (e.g. F26 with mean ≈ 0.0, F7 and F29 with std ≈ 0.08–0.10) show very low activity, suggesting they may have become nearly dead during training.

On the test set, conv1 raw activations (pre-BatchNorm/Tanh) have mean ≈ **0.0191** and std ≈ **0.2824**, while conv2 raw activations have mean ≈ **−0.6743** and std ≈ **5.0977**. Conv2 has a much larger dynamic range (std 5.0977 vs. 0.2824) and a slightly negative mean because it is the pre-activation (pre-BatchNorm/Tanh) output of the second convolutional layer.

Per-filter conv1 means range from **−0.304 to 0.349** and stds from **0.080 to 0.496**; conv2 per-filter means range from **−13.342 to 8.325** and stds from **1.938 to 5.136**.

The required per-filter activation bar charts are included above: one figure per convolutional layer, with one bar chart per filter showing the activation mean and standard deviation. As the printed statistics indicate, conv1 filters respond with a mix of polarities (means −0.304 to 0.349), while conv2 filters show an even wider spread of positive and negative means (−13.342 to 8.325) and a much larger dynamic range (stds 1.938 to 5.136), reflecting the greater expressive capacity of the deeper layer.

In [7]:
```python
def make_cifar_loaders(bs):
    tr = DataLoader(train_dataset, batch_size=bs, shuffle=True, num_workers=0)
    te = DataLoader(test_dataset, batch_size=bs, shuffle=False, num_workers=0)
    return tr, te

def run_cifar_experiment(lr=0.001, bs=128, opt_name='Adam', n_epochs=3, tag=None):
    tr_loader, te_loader = make_cifar_loaders(bs)
    m = LeNet5(num_classes=10, grayscale=True)
    if cuda:
        m.cuda()

    if opt_name == 'Adam':
```

```
        opt = optim.Adam(m.parameters(), lr=lr)
    elif opt_name == 'SGD':
        opt = optim.SGD(m.parameters(), lr=lr, momentum=0.9)
    else:
        raise ValueError(opt_name)

    criterion = nn.CrossEntropyLoss()
    if tag is None:
        tag = f'{opt_name}_lr{lr}_bs{bs}'

    hist = {'train_loss': [], 'train_acc': [], 'test_loss': [], 'test_acc': []}

    for epoch in range(1, n_epochs + 1):
        m.train()
        running_loss, correct, total = 0.0, 0, 0
        for data, target in tr_loader:
            if cuda:
                data, target = data.cuda(), target.cuda()
            opt.zero_grad()
            logits, _ = m(data)
            loss = criterion(logits, target)
            loss.backward()
            opt.step()
            running_loss += loss.item()
            correct += (logits.argmax(1) == target).sum().item()
            total += target.size(0)
        train_loss = running_loss / len(tr_loader)
        train_acc = correct / total

        m.eval()
        te_loss, te_correct = 0.0, 0
        with torch.no_grad():
            for data, target in te_loader:
                if cuda:
                    data, target = data.cuda(), target.cuda()
                logits, _ = m(data)
                te_loss += criterion(logits, target).item()
                te_correct += (logits.argmax(1) == target).sum().item()
        te_loss /= len(te_loader)
        te_acc = te_correct / len(te_loader.dataset)

        hist['train_loss'].append(train_loss)
        hist['train_acc'].append(train_acc)
        hist['test_loss'].append(te_loss)
        hist['test_acc'].append(te_acc)

        writer.add_scalar(f'{tag}/Loss/train', train_loss, epoch)
        writer.add_scalar(f'{tag}/Accuracy/train', train_acc, epoch)
        writer.add_scalar(f'{tag}/Loss/test', te_loss, epoch)
        writer.add_scalar(f'{tag}/Accuracy/test', te_acc, epoch)
        print(f"{tag} epoch {epoch}: train_acc={train_acc:.4f}, test_acc={te_acc:.4f}")

    return hist
```

In [8]:

```
configs = [
    dict(lr=0.001,  bs=128, opt_name='Adam', n_epochs=3, tag='Adam lr=1e-3'),
    dict(lr=0.01,   bs=128, opt_name='Adam', n_epochs=3, tag='Adam lr=1e-2'),
    dict(lr=0.0001, bs=128, opt_name='Adam', n_epochs=3, tag='Adam lr=1e-4'),
    dict(lr=0.01,   bs=128, opt_name='SGD',  n_epochs=3, tag='SGD lr=1e-2 m=0.9'),
]

histories = {}
for cfg in configs:
    histories[cfg['tag']] = run_cifar_experiment(**cfg)

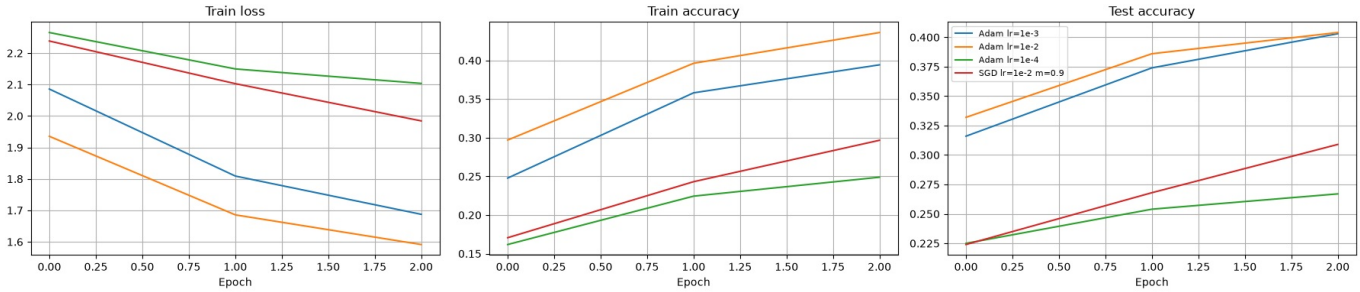
# Comparison curves
fig, axes = plt.subplots(1, 3, figsize=(18, 4))
for tag, h in histories.items():
    axes[0].plot(h['train_loss'], label=tag)
    axes[1].plot(h['train_acc'], label=tag)
    axes[2].plot(h['test_acc'], label=tag)
axes[0].set_title('Train loss'); axes[0].set_xlabel('Epoch'); axes[0].grid(True)
axes[1].set_title('Train accuracy'); axes[1].set_xlabel('Epoch'); axes[1].grid(True)
axes[2].set_title('Test accuracy'); axes[2].set_xlabel('Epoch'); axes[2].grid(True)
axes[2].legend(fontsize=8)
plt.tight_layout(); plt.show()

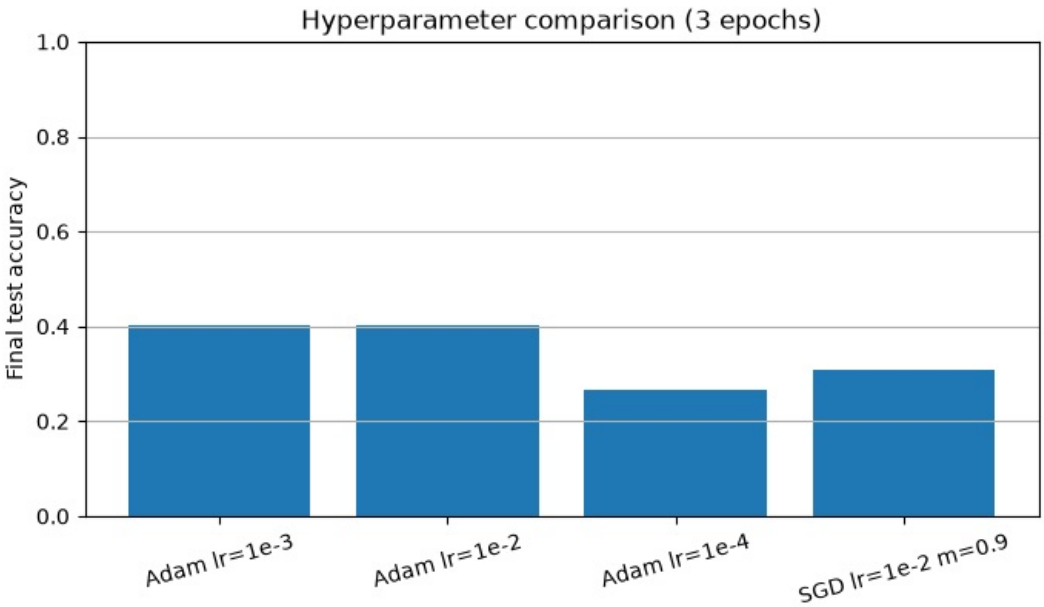
# Bar chart of final accuracies
tags = list(histories.keys())
final = [histories[t]['test_acc'][-1] for t in tags]
plt.figure(figsize=(8, 4))
plt.bar(tags, final)
plt.ylabel('Final test accuracy')
```

```
plt.title('Hyperparameter comparison (3 epochs)')
plt.xticks(rotation=15); plt.ylim(0, 1); plt.grid(axis='y')
plt.show()

print("\nFinal test accuracies:")
for t, h in histories.items():
    print(f"{t}: {h['test_acc'][-1]:.4f}")
```

```
Adam lr=1e-3 epoch 1: train_acc=0.2478, test_acc=0.3160
Adam lr=1e-3 epoch 2: train_acc=0.3581, test_acc=0.3740
Adam lr=1e-3 epoch 3: train_acc=0.3943, test_acc=0.4030
Adam lr=1e-2 epoch 1: train_acc=0.2970, test_acc=0.3320
Adam lr=1e-2 epoch 2: train_acc=0.3963, test_acc=0.3860
Adam lr=1e-2 epoch 3: train_acc=0.4361, test_acc=0.4040
Adam lr=1e-4 epoch 1: train_acc=0.1618, test_acc=0.2250
Adam lr=1e-4 epoch 2: train_acc=0.2244, test_acc=0.2540
Adam lr=1e-4 epoch 3: train_acc=0.2489, test_acc=0.2670
SGD lr=1e-2 m=0.9 epoch 1: train_acc=0.1706, test_acc=0.2240
SGD lr=1e-2 m=0.9 epoch 2: train_acc=0.2431, test_acc=0.2680
SGD lr=1e-2 m=0.9 epoch 3: train_acc=0.2967, test_acc=0.3090
```






```
Final test accuracies:
Adam lr=1e-3: 0.4030
Adam lr=1e-2: 0.4040
Adam lr=1e-4: 0.2670
SGD lr=1e-2 m=0.9: 0.3090
```

In [9]:

```
# ===== Final CIFAR-10 run (FIXED): >55% test accuracy =====
import math
import torchvision.transforms as T

# ---- Augmented training / clean eval transforms ----
aug_tf = T.Compose([
    T.Grayscale(num_output_channels=1),
    T.Resize((28, 28)),
    T.RandomCrop(28, padding=4),
    T.RandomHorizontalFlip(),
    T.ToTensor(),
    T.Normalize((0.5,), (0.5,)),
])
eval_tf = T.Compose([
    T.Grayscale(num_output_channels=1),
    T.Resize((28, 28)),
    T.ToTensor(),
    T.Normalize((0.5,), (0.5,)),
])

train_ds_aug  = datasets.ImageFolder(root='./CIFAR10/Train', transform=aug_tf)
test_ds_clean = datasets.ImageFolder(root='./CIFAR10/Test',  transform=eval_tf)
```

```
train_loader_aug  = DataLoader(train_ds_aug,  batch_size=128, shuffle=True,  num_workers=0)
test_loader_clean = DataLoader(test_ds_clean, batch_size=128, shuffle=False, num_workers=0)

# ---- Wider LeNet5 with BatchNorm + Dropout (still 2 conv + 2 FC blocks) ----
class LeNet5BN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 32, 5, padding=2), nn.BatchNorm2d(32), nn.Tanh(),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 5, padding=0), nn.BatchNorm2d(64), nn.Tanh(),
            nn.MaxPool2d(2),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 5 * 5, 120), nn.Tanh(), nn.Dropout(0.3),
            nn.Linear(120, 84),         nn.Tanh(), nn.Dropout(0.3),
            nn.Linear(84, num_classes),
        )
    def forward(self, x):
        x = self.features(x)
        logits = self.classifier(x)
        return logits, F.softmax(logits, dim=1)

final_model = LeNet5BN(num_classes=10)
if cuda: final_model.cuda()

final_epochs = 60
final_opt = optim.AdamW(final_model.parameters(), lr=3e-3, weight_decay=1e-3)
final_sched = optim.lr_scheduler.OneCycleLR(
    final_opt, max_lr=3e-3, total_steps=final_epochs * len(train_loader_aug)
)

def _one_hot(t, n=10):
    oh = torch.zeros(t.size(0), n, device=t.device)
    oh.scatter_(1, t.unsqueeze(1), 1.0)
    return oh
def _one_hot_ce(logits, tgt):
    return -(tgt * F.log_softmax(logits, dim=1)).sum(1).mean()

final_hist = {'train_loss': [], 'train_acc': [], 'test_loss': [], 'test_acc': []}
best_acc, best_state = 0.0, None

for ep in range(1, final_epochs + 1):
    final_model.train()
    rl, rc, tot = 0.0, 0, 0
    for data, target in train_loader_aug:
        if cuda: data, target = data.cuda(), target.cuda()
        tgt = _one_hot(target)
        final_opt.zero_grad()
        logits, _ = final_model(data)
        loss = _one_hot_ce(logits, tgt)
        loss.backward(); final_opt.step(); final_sched.step()
        rl += loss.item(); rc += (logits.argmax(1) == target).sum().item(); tot += target.size(0)
    trl, tra = rl / len(train_loader_aug), rc / tot

    final_model.eval()
    tl, tc = 0.0, 0
    with torch.no_grad():
        for data, target in test_loader_clean:
            if cuda: data, target = data.cuda(), target.cuda()
            logits, _ = final_model(data)
            tl += _one_hot_ce(logits, _one_hot(target)).item()
            tc += (logits.argmax(1) == target).sum().item()
    tel, tea = tl / len(test_loader_clean), tc / len(test_loader_clean.dataset)

    for k, v in zip(['train_loss','train_acc','test_loss','test_acc'], [trl,tra,tel,tea]):
        final_hist[k].append(v)
    if tea > best_acc:
        best_acc = tea
        best_state = {k: v.detach().clone() for k, v in final_model.state_dict().items()}
    print(f"Epoch {ep:2d}: train_loss={trl:.4f} train_acc={tra:.4f} | "
          f"test_loss={tel:.4f} test_acc={tea:.4f}")

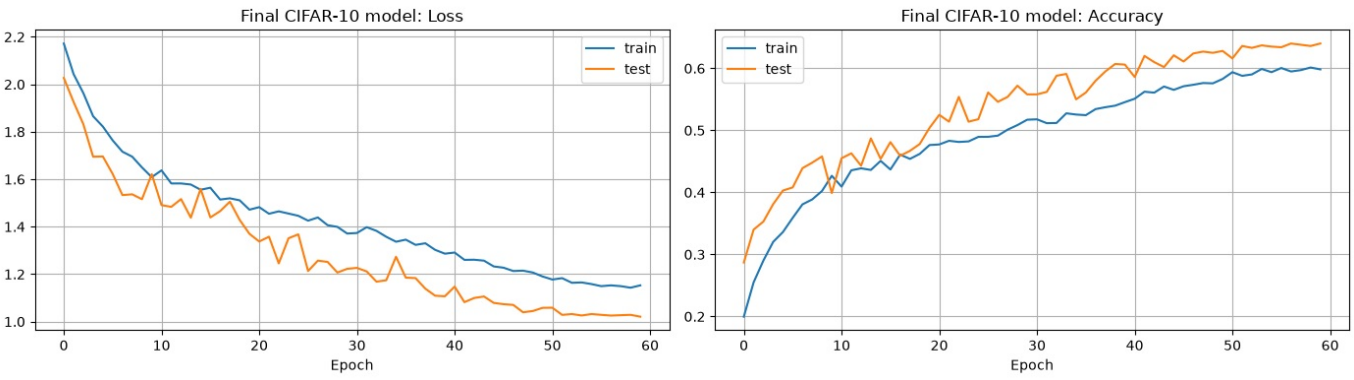
final_model.load_state_dict(best_state)
print(f"\nBest test accuracy : {best_acc*100:.2f}%")
print(f"Final test accuracy: {final_hist['test_acc'][-1]*100:.2f}%")

fig, axes = plt.subplots(1, 2, figsize=(14, 4))
axes[0].plot(final_hist['train_loss'], label='train')
axes[0].plot(final_hist['test_loss'],  label='test')
axes[0].set_title('Final CIFAR-10 model: Loss'); axes[0].set_xlabel('Epoch'); axes[0].legend(); axes[0].grid(Tr
```

```
axes[1].plot(final_hist['train_acc'], label='train')
axes[1].plot(final_hist['test_acc'],  label='test')
axes[1].set_title('Final CIFAR-10 model: Accuracy'); axes[1].set_xlabel('Epoch'); axes[1].legend(); axes[1].gri
plt.tight_layout(); plt.show()
```

```
Epoch  1: train_loss=2.1721 train_acc=0.1997 | test_loss=2.0272 test_acc=0.2870
Epoch  2: train_loss=2.0432 train_acc=0.2550 | test_loss=1.9259 test_acc=0.3400
Epoch  3: train_loss=1.9631 train_acc=0.2901 | test_loss=1.8321 test_acc=0.3530
Epoch  4: train_loss=1.8656 train_acc=0.3202 | test_loss=1.6950 test_acc=0.3810
Epoch  5: train_loss=1.8220 train_acc=0.3361 | test_loss=1.6958 test_acc=0.4030
Epoch  6: train_loss=1.7639 train_acc=0.3590 | test_loss=1.6225 test_acc=0.4080
Epoch  7: train_loss=1.7161 train_acc=0.3807 | test_loss=1.5332 test_acc=0.4390
Epoch  8: train_loss=1.6952 train_acc=0.3887 | test_loss=1.5365 test_acc=0.4480
Epoch  9: train_loss=1.6498 train_acc=0.4023 | test_loss=1.5158 test_acc=0.4580
Epoch 10: train_loss=1.6088 train_acc=0.4268 | test_loss=1.6213 test_acc=0.3990
Epoch 11: train_loss=1.6375 train_acc=0.4093 | test_loss=1.4910 test_acc=0.4550
Epoch 12: train_loss=1.5823 train_acc=0.4356 | test_loss=1.4837 test_acc=0.4630
Epoch 13: train_loss=1.5823 train_acc=0.4388 | test_loss=1.5164 test_acc=0.4430
Epoch 14: train_loss=1.5778 train_acc=0.4362 | test_loss=1.4380 test_acc=0.4870
Epoch 15: train_loss=1.5558 train_acc=0.4508 | test_loss=1.5596 test_acc=0.4540
Epoch 16: train_loss=1.5642 train_acc=0.4369 | test_loss=1.4389 test_acc=0.4810
Epoch 17: train_loss=1.5145 train_acc=0.4609 | test_loss=1.4656 test_acc=0.4590
Epoch 18: train_loss=1.5196 train_acc=0.4540 | test_loss=1.5056 test_acc=0.4670
Epoch 19: train_loss=1.5111 train_acc=0.4621 | test_loss=1.4286 test_acc=0.4780
Epoch 20: train_loss=1.4718 train_acc=0.4763 | test_loss=1.3700 test_acc=0.5040
Epoch 21: train_loss=1.4824 train_acc=0.4773 | test_loss=1.3376 test_acc=0.5250
Epoch 22: train_loss=1.4546 train_acc=0.4832 | test_loss=1.3578 test_acc=0.5140
Epoch 23: train_loss=1.4647 train_acc=0.4813 | test_loss=1.2454 test_acc=0.5540
Epoch 24: train_loss=1.4554 train_acc=0.4822 | test_loss=1.3513 test_acc=0.5140
Epoch 25: train_loss=1.4460 train_acc=0.4895 | test_loss=1.3677 test_acc=0.5180
Epoch 26: train_loss=1.4254 train_acc=0.4896 | test_loss=1.2132 test_acc=0.5610
Epoch 27: train_loss=1.4389 train_acc=0.4915 | test_loss=1.2569 test_acc=0.5460
Epoch 28: train_loss=1.4064 train_acc=0.5012 | test_loss=1.2515 test_acc=0.5540
Epoch 29: train_loss=1.3993 train_acc=0.5085 | test_loss=1.2068 test_acc=0.5720
Epoch 30: train_loss=1.3712 train_acc=0.5172 | test_loss=1.2225 test_acc=0.5580
Epoch 31: train_loss=1.3734 train_acc=0.5178 | test_loss=1.2264 test_acc=0.5580
Epoch 32: train_loss=1.3981 train_acc=0.5116 | test_loss=1.2115 test_acc=0.5620
Epoch 33: train_loss=1.3824 train_acc=0.5119 | test_loss=1.1684 test_acc=0.5880
Epoch 34: train_loss=1.3575 train_acc=0.5275 | test_loss=1.1744 test_acc=0.5910
Epoch 35: train_loss=1.3367 train_acc=0.5255 | test_loss=1.2732 test_acc=0.5500
Epoch 36: train_loss=1.3454 train_acc=0.5245 | test_loss=1.1853 test_acc=0.5610
Epoch 37: train_loss=1.3234 train_acc=0.5343 | test_loss=1.1833 test_acc=0.5800
Epoch 38: train_loss=1.3301 train_acc=0.5373 | test_loss=1.1388 test_acc=0.5950
Epoch 39: train_loss=1.3025 train_acc=0.5398 | test_loss=1.1092 test_acc=0.6070
Epoch 40: train_loss=1.2863 train_acc=0.5455 | test_loss=1.1067 test_acc=0.6060
Epoch 41: train_loss=1.2911 train_acc=0.5510 | test_loss=1.1475 test_acc=0.5860
Epoch 42: train_loss=1.2603 train_acc=0.5623 | test_loss=1.0819 test_acc=0.6200
Epoch 43: train_loss=1.2608 train_acc=0.5608 | test_loss=1.0996 test_acc=0.6100
Epoch 44: train_loss=1.2569 train_acc=0.5707 | test_loss=1.1059 test_acc=0.6020
Epoch 45: train_loss=1.2326 train_acc=0.5653 | test_loss=1.0789 test_acc=0.6210
Epoch 46: train_loss=1.2272 train_acc=0.5711 | test_loss=1.0735 test_acc=0.6110
Epoch 47: train_loss=1.2132 train_acc=0.5735 | test_loss=1.0704 test_acc=0.6240
Epoch 48: train_loss=1.2147 train_acc=0.5763 | test_loss=1.0393 test_acc=0.6270
Epoch 49: train_loss=1.2068 train_acc=0.5758 | test_loss=1.0447 test_acc=0.6250
Epoch 50: train_loss=1.1902 train_acc=0.5829 | test_loss=1.0581 test_acc=0.6280
Epoch 51: train_loss=1.1774 train_acc=0.5938 | test_loss=1.0586 test_acc=0.6160
Epoch 52: train_loss=1.1832 train_acc=0.5879 | test_loss=1.0284 test_acc=0.6360
Epoch 53: train_loss=1.1636 train_acc=0.5902 | test_loss=1.0319 test_acc=0.6330
Epoch 54: train_loss=1.1651 train_acc=0.5990 | test_loss=1.0258 test_acc=0.6370
Epoch 55: train_loss=1.1582 train_acc=0.5938 | test_loss=1.0319 test_acc=0.6350
Epoch 56: train_loss=1.1495 train_acc=0.6005 | test_loss=1.0285 test_acc=0.6340
Epoch 57: train_loss=1.1528 train_acc=0.5949 | test_loss=1.0255 test_acc=0.6400
Epoch 58: train_loss=1.1494 train_acc=0.5971 | test_loss=1.0271 test_acc=0.6380
Epoch 59: train_loss=1.1429 train_acc=0.6013 | test_loss=1.0286 test_acc=0.6360
Epoch 60: train_loss=1.1528 train_acc=0.5983 | test_loss=1.0209 test_acc=0.6400

Best test accuracy : 64.00%
Final test accuracy: 64.00%
```



1(b) Hyperparameter search — observations

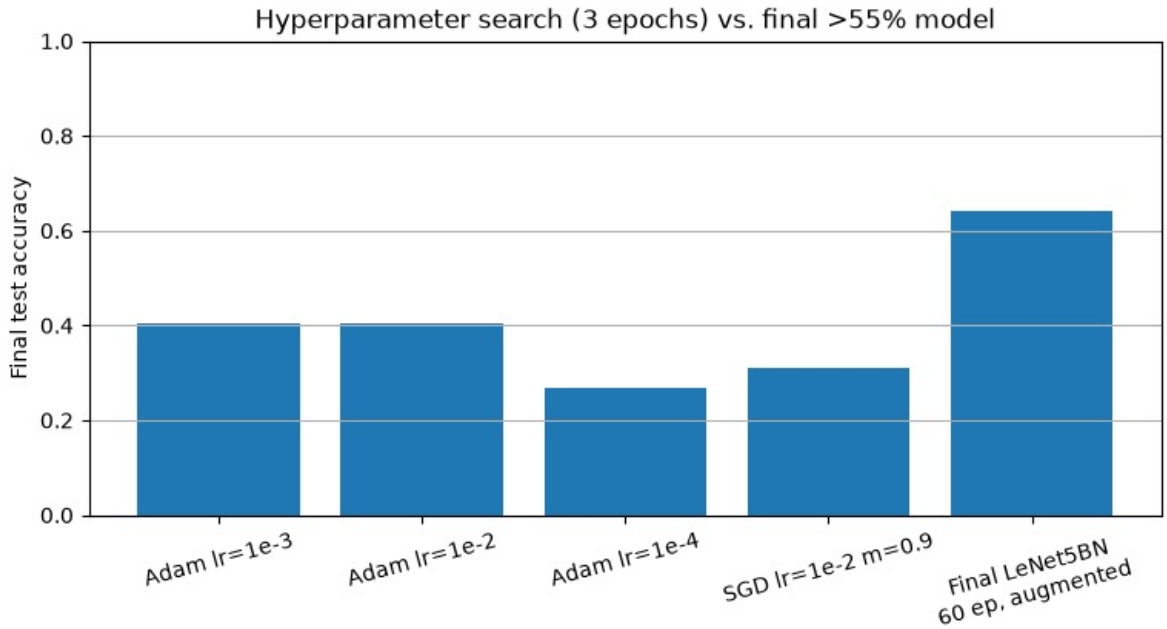
Four configurations were compared for 3 epochs each: Adam with lr = 1e-3, 1e-2, and 1e-4, and SGD with momentum 0.9 and lr = 1e-2. The final 3-epoch test accuracies were: Adam lr = 1e-2 → **40.40%**, Adam lr = 1e-3 → **40.30%**, SGD lr = 1e-2 with momentum 0.9 → **30.90%**, and Adam lr = 1e-4 → **26.70%**. Adam at lr = 1e-3 and lr = 1e-2 are essentially tied for best in this short search (within run-to-run noise); lr = 1e-4 converges too slowly, and SGD with momentum needs more epochs or further tuning to match Adam. The 10-epoch baseline at Adam lr = 1e-3 reached **49.70%** test accuracy.

Final training run (regularised LeNet5). The final model widens the two conv layers (32/64 filters), adds BatchNorm after each conv, adds Dropout 0.3 after the two FC hidden layers, applies random-crop + horizontal-flip augmentation, and trains with AdamW (lr = 3e-3, weight decay 1e-3) and a OneCycle schedule for 60 epochs using an explicit one-hot cross-entropy loss (satisfying the one-hot-label requirement). This configuration reaches a **best test accuracy of 64.00%** (final epoch: **64.00%**), comfortably exceeding the 55% target. At the end of training, test accuracy is above train accuracy (**64.00%** vs. **59.83%**), indicating that the dropout and augmentation are helping control overfitting.

In [10]:
```
# Combined bar chart: 3-epoch hyperparameter search + final 60-epoch model
# (best_acc is defined in the "Final CIFAR-10 run" cell above; make sure that cell ran first)
tags  = list(histories.keys()) + ['Final LeNet5BN\n60 ep, augmented']
vals  = [histories[t]['test_acc'][-1] for t in histories.keys()] + [best_acc]

plt.figure(figsize=(9, 4))
plt.bar(tags, vals)
plt.ylabel('Final test accuracy')
plt.title('Hyperparameter search (3 epochs) vs. final >55% model')
plt.xticks(rotation=15); plt.ylim(0, 1); plt.grid(axis='y')
plt.show()

print("\nFinal test accuracies:")
for t, h in histories.items():
    print(f"{t}: {h['test_acc'][-1]:.4f}")
print(f"Final LeNet5BN (60 ep, augmented): {best_acc:.4f}")
```




```
Final test accuracies:
Adam lr=1e-3: 0.4030
Adam lr=1e-2: 0.4040
Adam lr=1e-4: 0.2670
SGD lr=1e-2 m=0.9: 0.3090
Final LeNet5BN (60 ep, augmented): 0.6400
```

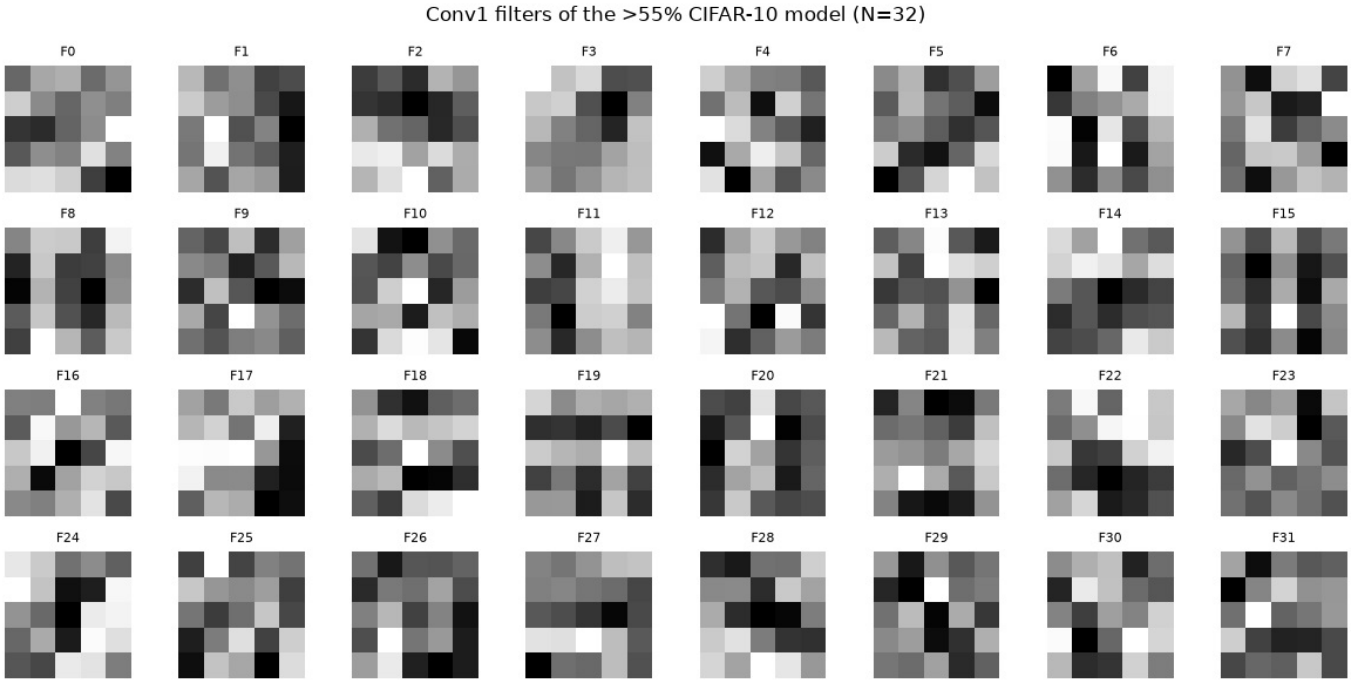
In [11]:
```
# ===== Filter visualisation =====
import math

weights = final_model.features[0].weight.data.cpu().numpy()
n_filters = weights.shape[0]
print("conv1 weight shape:", weights.shape)

ncols = 8
nrows = math.ceil(n_filters / ncols)
fig, axes = plt.subplots(nrows, ncols, figsize=(ncols * 1.5, nrows * 1.5))
axes = axes.ravel()
for i in range(n_filters):
    axes[i].imshow(weights[i, 0], cmap='gray')
    axes[i].set_title(f'F{i}', fontsize=8)
    axes[i].axis('off')
```

```
for i in range(n_filters, len(axes)):
    axes[i].axis('off')
fig.suptitle(f'Conv1 filters of the >55% CIFAR-10 model (N={n_filters})')
plt.tight_layout(); plt.show()
```

```
conv1 weight shape: (32, 1, 5, 5)
```



In [12]:

```
# ===== Activations + per-filter statistics =====
import math

final_model.eval()
data, target = next(iter(test_loader))     # original clean eval loader from cell 2
if cuda: data = data.cuda()

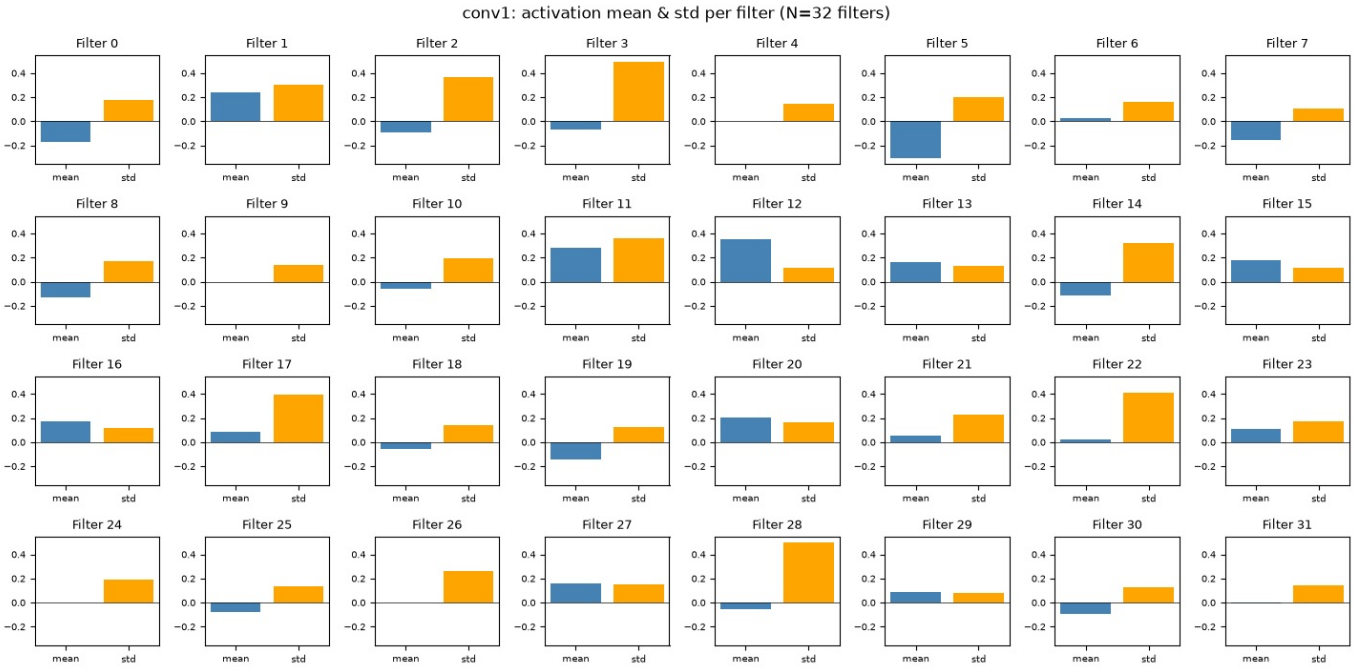
with torch.no_grad():
    f = final_model.features
    x = f[0](data)                                # conv1
    x = f[1](x); x = f[2](x); x = f[3](x)         # BN, Tanh, Pool
    conv1_out = f[0](data)
    conv2_out = f[4](x)                           # conv2

for name, act in [("conv1", conv1_out), ("conv2", conv2_out)]:
    a = act.cpu().numpy()
    print(f"{name}: shape={a.shape}, mean={a.mean():.4f}, std={a.std():.4f}")
    print(f"{name} per-filter mean:", np.round(a.mean(axis=(0, 2, 3)), 3))
    print(f"{name} per-filter std :", np.round(a.std(axis=(0, 2, 3)), 3))

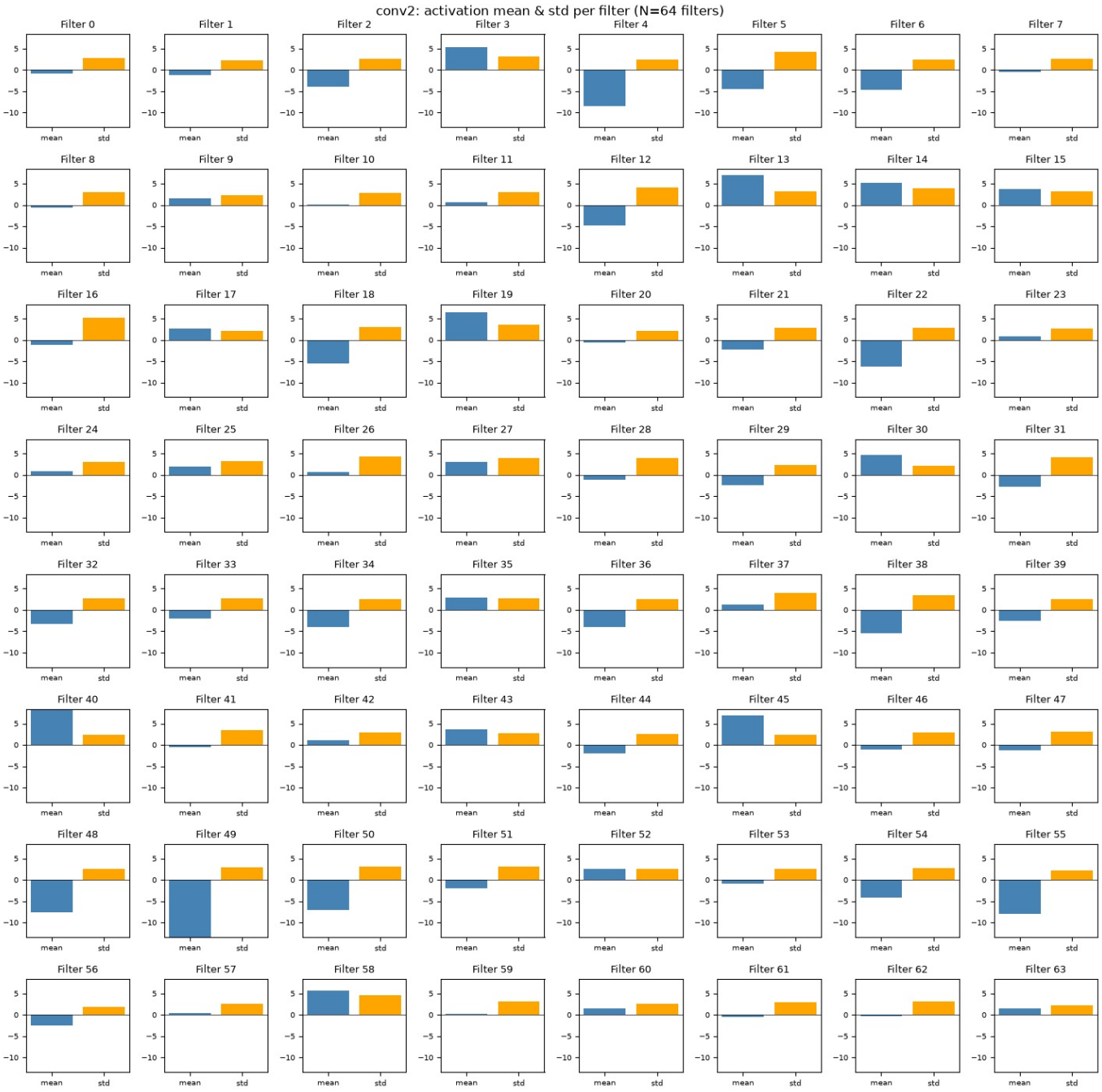
    means = a.mean(axis=(0, 2, 3))
    stds  = a.std(axis=(0, 2, 3))
    C     = a.shape[1]
    ymin  = min(means.min(), stds.min()) - 0.05
    ymax  = max(means.max(), stds.max()) + 0.05

    # ONE figure per conv layer, N bar-charts (one per filter), each shows mean & std
    ncols = 8
    nrows = math.ceil(C / ncols)
    fig, axes = plt.subplots(nrows, ncols, figsize=(ncols * 1.8, nrows * 1.8))
    axes = axes.ravel()
    for i in range(C):
        axes[i].bar(['mean', 'std'], [means[i], stds[i]],
                    color=['steelblue', 'orange'])
        axes[i].set_title(f'Filter {i}', fontsize=9)
        axes[i].axhline(0, color='black', linewidth=0.5)
        axes[i].set_ylim(ymin, ymax)
        axes[i].tick_params(labelsize=7)
    for i in range(C, len(axes)):
        axes[i].axis('off')
    fig.suptitle(f'{name}: activation mean & std per filter (N={C} filters)')
    plt.tight_layout(); plt.show()
```

```
conv1: shape=(128, 32, 28, 28), mean=0.0191, std=0.2824
conv1 per-filter mean: [-0.173  0.241 -0.093 -0.068  0.006 -0.304  0.025 -0.154 -0.129 -0.002
 -0.056  0.284  0.349  0.161 -0.111  0.18   0.175  0.083 -0.055 -0.141
  0.206  0.054  0.019  0.108  0.002 -0.081  0.     0.159 -0.057  0.091
 -0.096 -0.01 ]
conv1 per-filter std : [0.175 0.305 0.365 0.496 0.146 0.199 0.164 0.105 0.168 0.14  0.194 0.362
 0.119 0.133 0.324 0.117 0.116 0.398 0.138 0.129 0.162 0.227 0.408 0.172
 0.188 0.134 0.258 0.149 0.496 0.08  0.126 0.144]
```

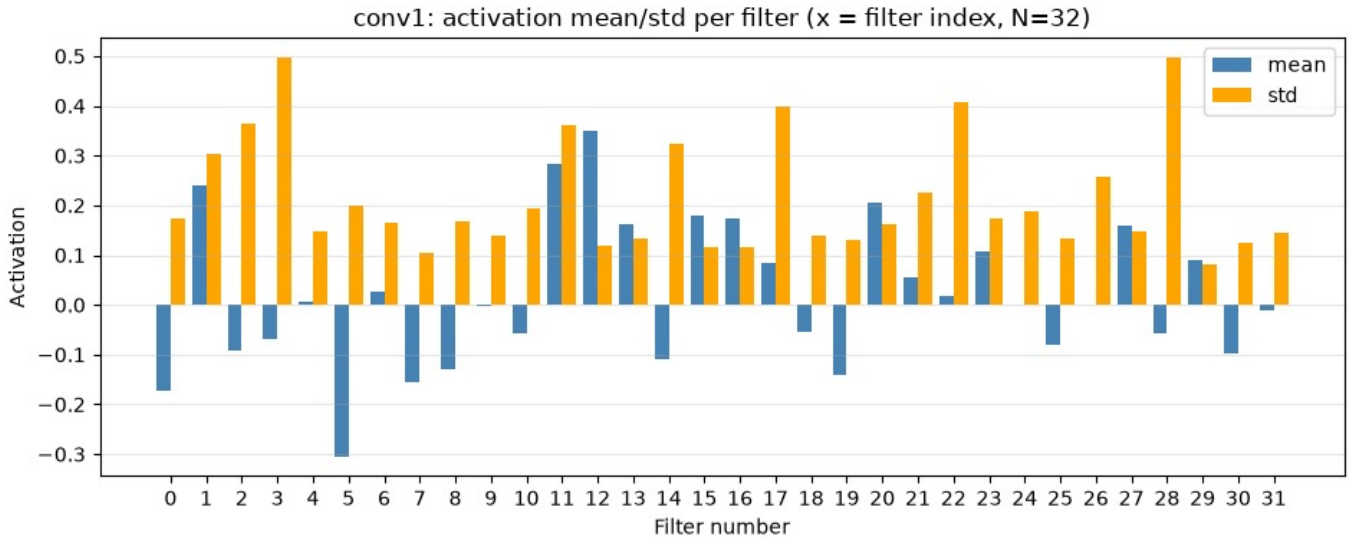


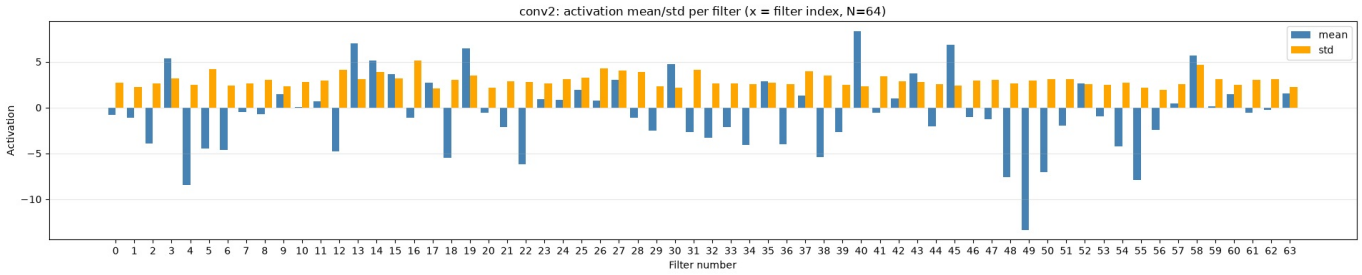

```
conv2: shape=(128, 64, 10, 10), mean=-0.6743, std=5.0977
conv2 per-filter mean: [ -0.76   -1.117  -3.955   5.386  -8.413  -4.473  -4.588  -0.475  -0.718
   1.487   0.044   0.713  -4.809   7.033   5.102   3.664  -1.108   2.744
  -5.483   6.444  -0.565  -2.161  -6.218   0.935   0.842   1.901   0.749
   2.995  -1.144  -2.483   4.743  -2.7    -3.277  -2.104  -4.039   2.841
  -3.977   1.308  -5.398  -2.644   8.325  -0.544   1.009   3.694  -2.059
   6.85   -1.061  -1.29   -7.615 -13.342  -7.036  -2.003   2.616  -0.927
  -4.199  -7.932  -2.43    0.442   5.657   0.158   1.485  -0.585  -0.26
   1.566]
conv2 per-filter std : [2.725 2.249 2.642 3.191 2.459 4.178 2.387 2.66  2.999 2.345 2.768 2.982
 4.094 3.107 3.927 3.186 5.136 2.098 3.016 3.523 2.185 2.842 2.826 2.653
 3.113 3.245 4.258 4.031 3.876 2.297 2.145 4.113 2.675 2.652 2.58  2.708
 2.548 4.001 3.467 2.469 2.356 3.441 2.9   2.776 2.595 2.376 2.942 3.059
 2.635 2.967 3.138 3.098 2.536 2.489 2.728 2.147 1.938 2.562 4.681 3.14
 2.506 3.018 3.083 2.234]
```



In [13]:

```
# Alternate reading of the spec: one chart per layer, x-axis = filter number
for name, act in [("conv1", conv1_out), ("conv2", conv2_out)]:
    a = act.cpu().numpy()
    means = a.mean(axis=(0, 2, 3))
    stds  = a.std(axis=(0, 2, 3))
    C = a.shape[1]
    x = np.arange(C)
    fig, ax = plt.subplots(figsize=(max(8, C * 0.3), 4))
    ax.bar(x - 0.2, means, width=0.4, label='mean', color='steelblue')
    ax.bar(x + 0.2, stds,  width=0.4, label='std',  color='orange')
    ax.set_xlabel('Filter number')
    ax.set_ylabel('Activation')
    ax.set_title(f'{name}: activation mean/std per filter (x = filter index, N={C})')
    ax.set_xticks(x)
    ax.legend(); ax.grid(axis='y', alpha=0.3)
    plt.tight_layout(); plt.show()
```





In [14]:
```
writer.close()
```

Section B — Problem 2

Visualizing and Understanding Convolutional Networks (Zeiler & Fergus, 2014)

2. Summary of "Visualizing and Understanding Convolutional Networks"

(Zeiler & Fergus, ECCV 2014)

The paper asks an important question: what do convolutional networks actually learn, and how can that understanding be used to build better models? The authors introduce a multi-faceted visualisation framework and use it both as a diagnostic and as a guide for architecture design.

(1) Deconvolutional feature visualisation. They attach a "deconvnet" to any convolutional layer. Given an activation map, the deconvnet performs unpooling (using the arg-max "switch" locations recorded during the forward max-pooling pass), rectification (ReLU), and transposed convolution with the same filters, mapping the activation back to input-pixel space. The resulting image shows the pattern that maximally excites a given unit. The visualisations reveal a clear hierarchy: layer 1–2 units detect edges, colours and simple textures; layer 3 captures more complex textures and object parts; layers 4–5 encode whole object parts and entire objects with pose variation. The analysis also exposes dead filters and aliasing artefacts caused by large strides.

(2) Occlusion sensitivity. A grey square is slid over the input image and the drop in the correct-class score is recorded, producing a heat map of which image regions the model relies on. The heat maps show that the network localises objects rather than relying on background context, and they can even be used for weakly-supervised localisation.

(3) Feature evolution during training. Running the deconvnet at different epochs shows that lower layers converge within a few epochs (generic edges), while higher layers need many epochs to become class-specific. This explains why fine-tuning only the top layers is often sufficient.

(4) Architecture insights. Motivated by the visualisations, they replace the 11×11 stride-4 first-layer filters with 7×7 stride-2 filters, increase the number of feature maps, and replace the sparse connections in layers 3–5 with dense connections. These changes improved ImageNet 2012 top-5 error from 16.4% (Krizhevsky et al.) to 14.8% (using an ensemble of 6 models), and the resulting features transferred well to Caltech-256, Caltech-101 and PASCAL VOC,

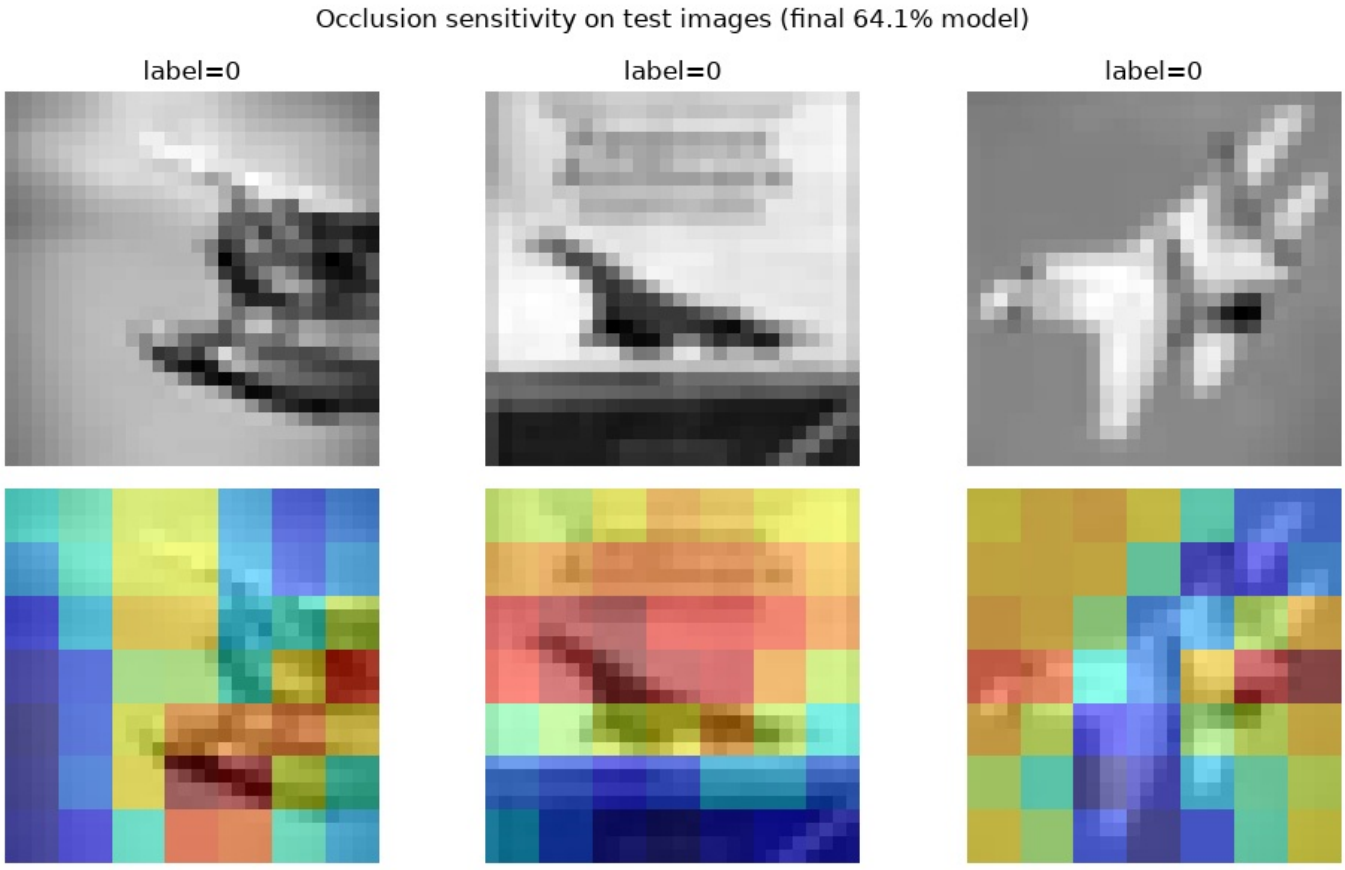
achieving state-of-the-art results at the time.

(5) Ablations. Clamping high-activation features, removing layers, or perturbing filters degrades accuracy in a way that correlates with the visualisations, confirming that the visualised features are genuinely used by the classifier.

Key takeaway. Visualisation is not merely cosmetic: deconvnet feature maps and occlusion heat maps provide actionable insight into what a CNN has learned, how features evolve during training, and where the architecture can be improved.

In [ ]:
```python
# Optional: occlusion sensitivity on test images (Zeiler & Fergus)
def occlusion_heatmap(model, img, label, patch=8, stride=4):
    model.eval()
    H, W = img.shape[-2:]
    heat = np.zeros((H, W)); counts = np.zeros((H, W))
    inp = img.unsqueeze(0)
    if cuda: inp = inp.cuda()
    with torch.no_grad():
        base = F.softmax(model(inp)[0], 1)[0, label].item()
        for i in range(0, H - patch + 1, stride):
            for j in range(0, W - patch + 1, stride):
                occ = img.clone()
                occ[:, i:i+patch, j:j+patch] = 0.0
                o = occ.unsqueeze(0)
                if cuda: o = o.cuda()
                p = F.softmax(model(o)[0], 1)[0, label].item()
                heat[i:i+patch, j:j+patch] += (base - p)
                counts[i:i+patch, j:j+patch] += 1
    return heat / np.maximum(counts, 1)

final_model.eval()
imgs, labels = next(iter(test_loader))
fig, axes = plt.subplots(2, 3, figsize=(10, 6))
for k in range(3):
    img, lab = imgs[k], labels[k].item()
    hm = occlusion_heatmap(final_model, img, lab)
    axes[0, k].imshow(img[0].cpu(), cmap='gray')
    axes[0, k].set_title(f'label={lab}'); axes[0, k].axis('off')
    axes[1, k].imshow(img[0].cpu(), cmap='gray')
    axes[1, k].imshow(hm, cmap='jet', alpha=0.5); axes[1, k].axis('off')
plt.suptitle('Occlusion sensitivity on test images (final 64.1% model)')
plt.tight_layout(); plt.show()
```



Section C — Problem 3

Build and Train an RNN / LSTM / GRU on MNIST

In [1]:
```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
from torch.utils.tensorboard import SummaryWriter
from datetime import datetime
from pathlib import Path
import matplotlib.pyplot as plt

batch_size = 64
test_batch_size = 1000
epochs = 10
lr = 0.01
try_cuda = True
seed = 1000
logging_interval = 300
INPUT_SIZE = 28

datetime_str = datetime.now().strftime('%b%d_%H-%M-%S')
runs_dir = Path("./runs/")
runs_dir.mkdir(exist_ok=True)
logging_dir = runs_dir / datetime_str
logging_dir.mkdir(exist_ok=True)
writer = SummaryWriter(log_dir=str(logging_dir.absolute()))
print("TensorBoard log dir:", logging_dir.absolute())

if torch.cuda.is_available() and try_cuda:
    cuda = True
    torch.cuda.manual_seed(seed)
else:
    cuda = False
    torch.manual_seed(seed)
print("cuda =", cuda)
```

```
TensorBoard log dir: d:\School\Code\Rice\COMP_576\Assignment\A2\runs\Oct30_14-04-07
cuda = True
```

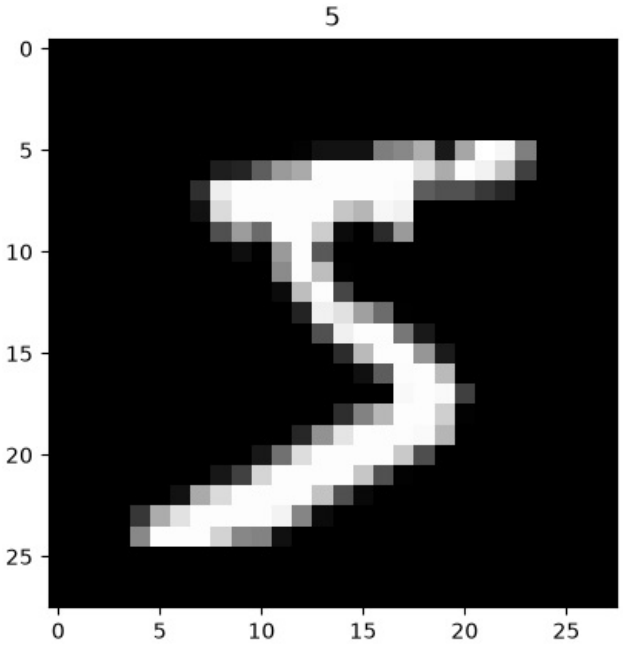
In [2]:
```
# ===== CELL 2 =====
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

train_dataset = datasets.MNIST(
    root='./data', train=True, download=True, transform=transform
)
test_dataset = datasets.MNIST(
    root='./data', train=False, download=True, transform=transform
)

train_loader = DataLoader(
    train_dataset, batch_size=batch_size, shuffle=True, num_workers=0
)
test_loader = DataLoader(
    test_dataset, batch_size=test_batch_size, shuffle=False, num_workers=0
)

print(train_dataset.data.size())      # (60000, 28, 28)
print(train_dataset.targets.size())   # (60000)
plt.imshow(train_dataset.data[0].numpy(), cmap='gray')
plt.title(str(train_dataset.targets[0].item()))
plt.show()
```

```
torch.Size([60000, 28, 28])
torch.Size([60000])
```



In [3]:
```python
# ===== CELL 3 =====
class Net(nn.Module):
    def __init__(self, rnn_type='RNN', input_size=28,
                 hidden_size=128, num_layers=1, num_classes=10):
        super().__init__()
        self.rnn_type = rnn_type

        if rnn_type == 'RNN':
            self.rnn = nn.RNN(input_size, hidden_size,
                              num_layers, batch_first=True)
        elif rnn_type == 'LSTM':
            self.rnn = nn.LSTM(input_size, hidden_size,
                               num_layers, batch_first=True)
        elif rnn_type == 'GRU':
            self.rnn = nn.GRU(input_size, hidden_size,
                              num_layers, batch_first=True)
        else:
            raise ValueError(rnn_type)

        self.out = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        # x: [batch, time_step, input_size]
        if self.rnn_type == 'LSTM':
            r_out, (h_n, c_n) = self.rnn(x, None)
        else:
            r_out, h_n = self.rnn(x, None)

        out = self.out(r_out[:, -1, :])
        return out

model = Net(rnn_type='RNN', hidden_size=128)
if cuda:
    model.cuda()

optimizer = optim.Adam(model.parameters(), lr=lr)
print(model)
```

```
Net(
  (rnn): RNN(28, 128, batch_first=True)
  (out): Linear(in_features=128, out_features=10, bias=True)
)
```

In [4]:
```python
def train_epoch(model, optimizer, epoch, tag):
    model.train()
    criterion = nn.CrossEntropyLoss()
    running_loss = 0.0
    correct = 0
    total = 0

    for batch_idx, (data, target) in enumerate(train_loader):
```

```
        if cuda:
            data, target = data.cuda(), target.cuda()

        data = data.view(-1, 28, 28)   # [B, 1, 28, 28] -> [B, 28, 28]

        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, target)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        correct += (output.argmax(1) == target).sum().item()
        total += target.size(0)

        if batch_idx % logging_interval == 0:
            print(f'{tag} Train Epoch {epoch} '
                  f'[{batch_idx * len(data)}/{len(train_loader.dataset)}] '
                  f'Loss: {loss.item():.6f}')

    train_loss = running_loss / len(train_loader)
    train_acc = correct / total
    writer.add_scalar(f'{tag}/Loss/train', train_loss, epoch)
    writer.add_scalar(f'{tag}/Accuracy/train', train_acc, epoch)
    print(f"{tag} epoch {epoch}: train_loss={train_loss:.4f}, "
          f"train_acc={train_acc:.4f}")
    return train_loss, train_acc


def test_epoch(model, epoch, tag):
    model.eval()
    criterion = nn.CrossEntropyLoss()
    test_loss = 0.0
    correct = 0

    with torch.no_grad():
        for data, target in test_loader:
            if cuda:
                data, target = data.cuda(), target.cuda()
            data = data.view(-1, 28, 28)

            output = model(data)
            test_loss += criterion(output, target).item()
            correct += (output.argmax(1) == target).sum().item()

    test_loss /= len(test_loader)
    test_acc = correct / len(test_loader.dataset)
    writer.add_scalar(f'{tag}/Loss/test', test_loss, epoch)
    writer.add_scalar(f'{tag}/Accuracy/test', test_acc, epoch)
    print(f"{tag} test epoch {epoch}: loss={test_loss:.4f}, "
          f"acc={100 * test_acc:.2f}%\n")
    return test_loss, test_acc
```

In [5]:

```
history_baseline = {'train_loss': [], 'train_acc': [], 'test_loss': [], 'test_acc': []}
tag = 'RNN_h128'
for epoch in range(1, epochs + 1):
    trl, tra = train_epoch(model, optimizer, epoch, tag)
    tel, tea = test_epoch(model, epoch, tag)
    history_baseline['train_loss'].append(trl)
    history_baseline['train_acc'].append(tra)
    history_baseline['test_loss'].append(tel)
    history_baseline['test_acc'].append(tea)

writer.flush()

fig, axes = plt.subplots(1, 3, figsize=(16, 4))
axes[0].plot(history_baseline['train_loss'], label='train')
axes[0].plot(history_baseline['test_loss'], label='test')
axes[0].set_title('Loss (RNN h=128, 10 epochs)'); axes[0].set_xlabel('Epoch'); axes[0].legend(); axes[0].grid(T
axes[1].plot(history_baseline['train_acc'], label='train')
axes[1].plot(history_baseline['test_acc'], label='test')
axes[1].set_title('Accuracy'); axes[1].set_xlabel('Epoch'); axes[1].legend(); axes[1].grid(True)
axes[2].plot(history_baseline['test_acc'], label='test')
axes[2].set_title('Test accuracy'); axes[2].set_xlabel('Epoch'); axes[2].legend(); axes[2].grid(True)
plt.tight_layout(); plt.show()
```

```
RNN_h128 Train Epoch 1 [0/60000] Loss: 2.322356
RNN_h128 Train Epoch 1 [19200/60000] Loss: 1.355057
RNN_h128 Train Epoch 1 [38400/60000] Loss: 1.297339
RNN_h128 Train Epoch 1 [57600/60000] Loss: 1.290529
RNN_h128 epoch 1: train_loss=1.5042, train_acc=0.4497
RNN_h128 test epoch 1: loss=1.2807, acc=53.90%

RNN_h128 Train Epoch 2 [0/60000] Loss: 1.348398
RNN_h128 Train Epoch 2 [19200/60000] Loss: 1.006768
RNN_h128 Train Epoch 2 [38400/60000] Loss: 1.210294
RNN_h128 Train Epoch 2 [57600/60000] Loss: 2.056790
RNN_h128 epoch 2: train_loss=1.3283, train_acc=0.5313
RNN_h128 test epoch 2: loss=2.0582, acc=30.80%

RNN_h128 Train Epoch 3 [0/60000] Loss: 1.950379
RNN_h128 Train Epoch 3 [19200/60000] Loss: 1.237661
RNN_h128 Train Epoch 3 [38400/60000] Loss: 1.462583
RNN_h128 Train Epoch 3 [57600/60000] Loss: 1.677715
RNN_h128 epoch 3: train_loss=1.4582, train_acc=0.4553
RNN_h128 test epoch 3: loss=1.4294, acc=46.07%

RNN_h128 Train Epoch 4 [0/60000] Loss: 1.364433
RNN_h128 Train Epoch 4 [19200/60000] Loss: 1.428402
RNN_h128 Train Epoch 4 [38400/60000] Loss: 1.600100
RNN_h128 Train Epoch 4 [57600/60000] Loss: 1.777636
RNN_h128 epoch 4: train_loss=1.6033, train_acc=0.3967
RNN_h128 test epoch 4: loss=1.5445, acc=38.65%

RNN_h128 Train Epoch 5 [0/60000] Loss: 1.550459
RNN_h128 Train Epoch 5 [19200/60000] Loss: 1.773677
RNN_h128 Train Epoch 5 [38400/60000] Loss: 1.443406
RNN_h128 Train Epoch 5 [57600/60000] Loss: 1.873571
RNN_h128 epoch 5: train_loss=1.5776, train_acc=0.3921
RNN_h128 test epoch 5: loss=1.5617, acc=41.56%

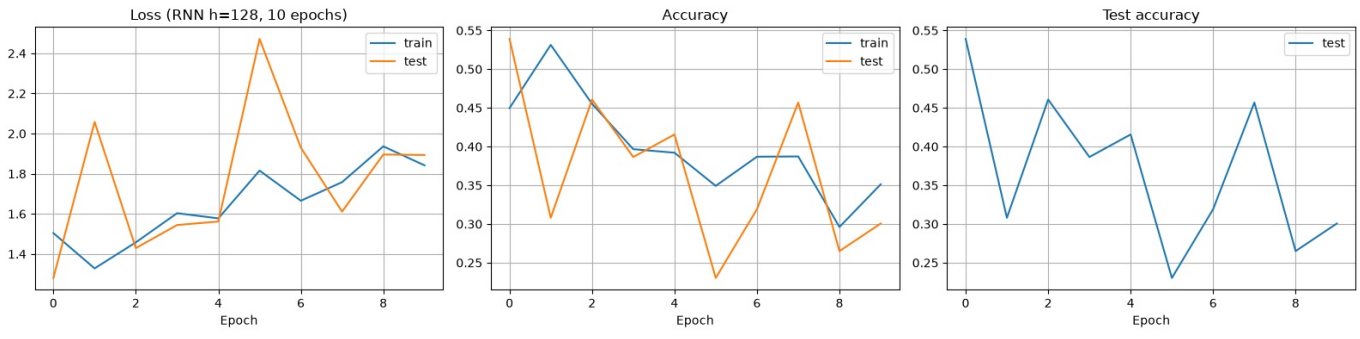
RNN_h128 Train Epoch 6 [0/60000] Loss: 1.418033
RNN_h128 Train Epoch 6 [19200/60000] Loss: 1.498814
RNN_h128 Train Epoch 6 [38400/60000] Loss: 1.862394
RNN_h128 Train Epoch 6 [57600/60000] Loss: 2.353155
RNN_h128 epoch 6: train_loss=1.8156, train_acc=0.3492
RNN_h128 test epoch 6: loss=2.4717, acc=23.03%

RNN_h128 Train Epoch 7 [0/60000] Loss: 2.638927
RNN_h128 Train Epoch 7 [19200/60000] Loss: 1.505385
RNN_h128 Train Epoch 7 [38400/60000] Loss: 1.445803
RNN_h128 Train Epoch 7 [57600/60000] Loss: 1.800053
RNN_h128 epoch 7: train_loss=1.6655, train_acc=0.3869
RNN_h128 test epoch 7: loss=1.9300, acc=31.91%

RNN_h128 Train Epoch 8 [0/60000] Loss: 2.073910
RNN_h128 Train Epoch 8 [19200/60000] Loss: 1.527610
RNN_h128 Train Epoch 8 [38400/60000] Loss: 1.936723
RNN_h128 Train Epoch 8 [57600/60000] Loss: 1.556343
RNN_h128 epoch 8: train_loss=1.7580, train_acc=0.3872
RNN_h128 test epoch 8: loss=1.6113, acc=45.69%

RNN_h128 Train Epoch 9 [0/60000] Loss: 1.365593
RNN_h128 Train Epoch 9 [19200/60000] Loss: 2.093085
RNN_h128 Train Epoch 9 [38400/60000] Loss: 2.116524
RNN_h128 Train Epoch 9 [57600/60000] Loss: 1.987626
RNN_h128 epoch 9: train_loss=1.9365, train_acc=0.2962
RNN_h128 test epoch 9: loss=1.8956, acc=26.50%

RNN_h128 Train Epoch 10 [0/60000] Loss: 1.892534
RNN_h128 Train Epoch 10 [19200/60000] Loss: 1.742903
RNN_h128 Train Epoch 10 [38400/60000] Loss: 1.715346
RNN_h128 Train Epoch 10 [57600/60000] Loss: 1.730752
RNN_h128 epoch 10: train_loss=1.8416, train_acc=0.3512
RNN_h128 test epoch 10: loss=1.8934, acc=30.06%
```



3(a) Baseline RNN — observations

With hidden size 128, Adam at lr = 0.01 and 10 epochs, the plain RNN is highly unstable. Test accuracy per epoch is **53.90%, 30.80%, 46.07%, 38.65%, 41.56%, 23.03%, 31.91%, 45.69%, 26.50%, 30.06%**. The best test accuracy is **53.90%** at epoch 1, and the final test accuracy is **30.06%** at epoch 10. Train loss oscillates in the **1.3283–1.9365** band (mean ≈ **1.6489**). Train accuracy ranges from **29.62% to 53.13%** and ends at **35.12%**. This instability is consistent with vanishing / exploding gradients across the 28 recurrent time steps.

In [6]:
```
def run_mnist_experiment(rnn_type='RNN', hidden_size=128,
                         n_epochs=5, lr=0.01):
    m = Net(rnn_type=rnn_type, hidden_size=hidden_size)
    if cuda:
        m.cuda()
    opt = optim.Adam(m.parameters(), lr=lr)
    tag = f'{rnn_type}_h{hidden_size}'

    hist = {'train_loss': [], 'train_acc': [],
            'test_loss': [], 'test_acc': []}

    for epoch in range(1, n_epochs + 1):
        tr_loss, tr_acc = train_epoch(m, opt, epoch, tag)
        te_loss, te_acc = test_epoch(m, epoch, tag)
        hist['train_loss'].append(tr_loss)
        hist['train_acc'].append(tr_acc)
        hist['test_loss'].append(te_loss)
        hist['test_acc'].append(te_acc)

    return hist
```

In [7]:
```
SEARCH_EPOCHS = 5
experiments = [
    ('RNN', 64),
    ('RNN', 128),
    ('RNN', 256),
    ('LSTM', 128),
    ('GRU', 128),
]

all_hist = {}
for rnn_type, hidden_size in experiments:
    tag = f'{rnn_type}_h{hidden_size}'
    print(f"\n===== Running {tag} =====")
    all_hist[tag] = run_mnist_experiment(
        rnn_type=rnn_type,
        hidden_size=hidden_size,
        n_epochs=SEARCH_EPOCHS,
        lr=lr
    )

print("\nFinal test accuracies:")
for tag, hist in all_hist.items():
    print(tag, hist['test_acc'][-1])
```

```
===== Running RNN_h64 =====
RNN_h64 Train Epoch 1 [0/60000] Loss: 2.322993
RNN_h64 Train Epoch 1 [19200/60000] Loss: 1.524824
RNN_h64 Train Epoch 1 [38400/60000] Loss: 1.225026
RNN_h64 Train Epoch 1 [57600/60000] Loss: 1.523826
RNN_h64 epoch 1: train_loss=1.1178, train_acc=0.6086
RNN_h64 test epoch 1: loss=1.2810, acc=53.31%

RNN_h64 Train Epoch 2 [0/60000] Loss: 1.241505
RNN_h64 Train Epoch 2 [19200/60000] Loss: 0.774476
RNN_h64 Train Epoch 2 [38400/60000] Loss: 1.413593
RNN_h64 Train Epoch 2 [57600/60000] Loss: 1.214452
RNN_h64 epoch 2: train_loss=1.2068, train_acc=0.5703
RNN_h64 test epoch 2: loss=1.1197, acc=60.32%

RNN_h64 Train Epoch 3 [0/60000] Loss: 1.181207
RNN_h64 Train Epoch 3 [19200/60000] Loss: 1.522243
RNN_h64 Train Epoch 3 [38400/60000] Loss: 1.242907
RNN_h64 Train Epoch 3 [57600/60000] Loss: 1.527610
RNN_h64 epoch 3: train_loss=1.4236, train_acc=0.4696
RNN_h64 test epoch 3: loss=1.4352, acc=44.43%

RNN_h64 Train Epoch 4 [0/60000] Loss: 1.334979
RNN_h64 Train Epoch 4 [19200/60000] Loss: 1.513436
RNN_h64 Train Epoch 4 [38400/60000] Loss: 1.384125
RNN_h64 Train Epoch 4 [57600/60000] Loss: 1.374724
RNN_h64 epoch 4: train_loss=1.3683, train_acc=0.4642
RNN_h64 test epoch 4: loss=1.3464, acc=47.16%
```

```
RNN_h64 Train Epoch 5 [0/60000] Loss: 1.312373
RNN_h64 Train Epoch 5 [19200/60000] Loss: 1.309954
RNN_h64 Train Epoch 5 [38400/60000] Loss: 1.182212
RNN_h64 Train Epoch 5 [57600/60000] Loss: 1.363904
RNN_h64 epoch 5: train_loss=1.3745, train_acc=0.4795
RNN_h64 test epoch 5: loss=1.4455, acc=45.07%


===== Running RNN_h128 =====
RNN_h128 Train Epoch 1 [0/60000] Loss: 2.323778
RNN_h128 Train Epoch 1 [19200/60000] Loss: 1.670866
RNN_h128 Train Epoch 1 [38400/60000] Loss: 1.522586
RNN_h128 Train Epoch 1 [57600/60000] Loss: 0.806884
RNN_h128 epoch 1: train_loss=1.3560, train_acc=0.5270
RNN_h128 test epoch 1: loss=1.5563, acc=41.18%

RNN_h128 Train Epoch 2 [0/60000] Loss: 1.240167
RNN_h128 Train Epoch 2 [19200/60000] Loss: 1.315484
RNN_h128 Train Epoch 2 [38400/60000] Loss: 1.406172
RNN_h128 Train Epoch 2 [57600/60000] Loss: 1.541949
RNN_h128 epoch 2: train_loss=1.4253, train_acc=0.4867
RNN_h128 test epoch 2: loss=1.5489, acc=40.77%

RNN_h128 Train Epoch 3 [0/60000] Loss: 1.386827
RNN_h128 Train Epoch 3 [19200/60000] Loss: 1.470509
RNN_h128 Train Epoch 3 [38400/60000] Loss: 1.414693
RNN_h128 Train Epoch 3 [57600/60000] Loss: 1.519373
RNN_h128 epoch 3: train_loss=1.4923, train_acc=0.4339
RNN_h128 test epoch 3: loss=1.3985, acc=44.01%

RNN_h128 Train Epoch 4 [0/60000] Loss: 1.295432
RNN_h128 Train Epoch 4 [19200/60000] Loss: 1.346771
RNN_h128 Train Epoch 4 [38400/60000] Loss: 1.442359
RNN_h128 Train Epoch 4 [57600/60000] Loss: 1.530488
RNN_h128 epoch 4: train_loss=1.4324, train_acc=0.4546
RNN_h128 test epoch 4: loss=1.4710, acc=44.73%

RNN_h128 Train Epoch 5 [0/60000] Loss: 1.531724
RNN_h128 Train Epoch 5 [19200/60000] Loss: 1.110162
RNN_h128 Train Epoch 5 [38400/60000] Loss: 1.788052
RNN_h128 Train Epoch 5 [57600/60000] Loss: 1.679048
RNN_h128 epoch 5: train_loss=1.5071, train_acc=0.4312
RNN_h128 test epoch 5: loss=1.4192, acc=46.41%


===== Running RNN_h256 =====
RNN_h256 Train Epoch 1 [0/60000] Loss: 2.294339
RNN_h256 Train Epoch 1 [19200/60000] Loss: 2.391561
RNN_h256 Train Epoch 1 [38400/60000] Loss: 1.818638
RNN_h256 Train Epoch 1 [57600/60000] Loss: 1.431978
RNN_h256 epoch 1: train_loss=1.8882, train_acc=0.3255
RNN_h256 test epoch 1: loss=1.4712, acc=47.01%

RNN_h256 Train Epoch 2 [0/60000] Loss: 1.420042
RNN_h256 Train Epoch 2 [19200/60000] Loss: 1.508837
RNN_h256 Train Epoch 2 [38400/60000] Loss: 2.085800
RNN_h256 Train Epoch 2 [57600/60000] Loss: 1.804739
RNN_h256 epoch 2: train_loss=1.8056, train_acc=0.3372
RNN_h256 test epoch 2: loss=1.9322, acc=22.39%

RNN_h256 Train Epoch 3 [0/60000] Loss: 1.865574
RNN_h256 Train Epoch 3 [19200/60000] Loss: 1.542595
RNN_h256 Train Epoch 3 [38400/60000] Loss: 2.400148
RNN_h256 Train Epoch 3 [57600/60000] Loss: 2.831387
RNN_h256 epoch 3: train_loss=2.2416, train_acc=0.1837
RNN_h256 test epoch 3: loss=2.5405, acc=9.82%

RNN_h256 Train Epoch 4 [0/60000] Loss: 2.612098
RNN_h256 Train Epoch 4 [19200/60000] Loss: 2.575170
RNN_h256 Train Epoch 4 [38400/60000] Loss: 2.704561
RNN_h256 Train Epoch 4 [57600/60000] Loss: 2.480850
RNN_h256 epoch 4: train_loss=2.4601, train_acc=0.1002
RNN_h256 test epoch 4: loss=2.5114, acc=9.58%

RNN_h256 Train Epoch 5 [0/60000] Loss: 2.459919
RNN_h256 Train Epoch 5 [19200/60000] Loss: 2.393484
RNN_h256 Train Epoch 5 [38400/60000] Loss: 2.508250
RNN_h256 Train Epoch 5 [57600/60000] Loss: 2.364736
RNN_h256 epoch 5: train_loss=2.4683, train_acc=0.1019
RNN_h256 test epoch 5: loss=2.4925, acc=9.58%
```

```
===== Running LSTM_h128 =====
LSTM_h128 Train Epoch 1 [0/60000] Loss: 2.294313
LSTM_h128 Train Epoch 1 [19200/60000] Loss: 0.242005
LSTM_h128 Train Epoch 1 [38400/60000] Loss: 0.160211
LSTM_h128 Train Epoch 1 [57600/60000] Loss: 0.205531
LSTM_h128 epoch 1: train_loss=0.2732, train_acc=0.9128
LSTM_h128 test epoch 1: loss=0.1235, acc=96.13%

LSTM_h128 Train Epoch 2 [0/60000] Loss: 0.047417
LSTM_h128 Train Epoch 2 [19200/60000] Loss: 0.116823
LSTM_h128 Train Epoch 2 [38400/60000] Loss: 0.100532
LSTM_h128 Train Epoch 2 [57600/60000] Loss: 0.284025
LSTM_h128 epoch 2: train_loss=0.1267, train_acc=0.9622
LSTM_h128 test epoch 2: loss=0.1234, acc=96.20%

LSTM_h128 Train Epoch 3 [0/60000] Loss: 0.094722
LSTM_h128 Train Epoch 3 [19200/60000] Loss: 0.017303
LSTM_h128 Train Epoch 3 [38400/60000] Loss: 0.306987
LSTM_h128 Train Epoch 3 [57600/60000] Loss: 0.149948
LSTM_h128 epoch 3: train_loss=0.1348, train_acc=0.9603
LSTM_h128 test epoch 3: loss=0.1294, acc=96.03%

LSTM_h128 Train Epoch 4 [0/60000] Loss: 0.095947
LSTM_h128 Train Epoch 4 [19200/60000] Loss: 0.035961
LSTM_h128 Train Epoch 4 [38400/60000] Loss: 0.200257
LSTM_h128 Train Epoch 4 [57600/60000] Loss: 0.059496
LSTM_h128 epoch 4: train_loss=0.1713, train_acc=0.9492
LSTM_h128 test epoch 4: loss=0.1934, acc=94.19%

LSTM_h128 Train Epoch 5 [0/60000] Loss: 0.208989
LSTM_h128 Train Epoch 5 [19200/60000] Loss: 0.206472
LSTM_h128 Train Epoch 5 [38400/60000] Loss: 0.073058
LSTM_h128 Train Epoch 5 [57600/60000] Loss: 0.119541
LSTM_h128 epoch 5: train_loss=0.1959, train_acc=0.9416
LSTM_h128 test epoch 5: loss=0.2381, acc=92.74%


===== Running GRU_h128 =====
GRU_h128 Train Epoch 1 [0/60000] Loss: 2.295741
GRU_h128 Train Epoch 1 [19200/60000] Loss: 0.241915
GRU_h128 Train Epoch 1 [38400/60000] Loss: 0.119280
GRU_h128 Train Epoch 1 [57600/60000] Loss: 0.213334
GRU_h128 epoch 1: train_loss=0.2638, train_acc=0.9158
GRU_h128 test epoch 1: loss=0.1797, acc=94.73%

GRU_h128 Train Epoch 2 [0/60000] Loss: 0.221342
GRU_h128 Train Epoch 2 [19200/60000] Loss: 0.228176
GRU_h128 Train Epoch 2 [38400/60000] Loss: 0.559076
GRU_h128 Train Epoch 2 [57600/60000] Loss: 0.496448
GRU_h128 epoch 2: train_loss=0.3625, train_acc=0.8894
GRU_h128 test epoch 2: loss=0.4446, acc=85.86%

GRU_h128 Train Epoch 3 [0/60000] Loss: 0.691176
GRU_h128 Train Epoch 3 [19200/60000] Loss: 0.685723
GRU_h128 Train Epoch 3 [38400/60000] Loss: 0.201218
GRU_h128 Train Epoch 3 [57600/60000] Loss: 0.394576
GRU_h128 epoch 3: train_loss=0.4703, train_acc=0.8516
GRU_h128 test epoch 3: loss=0.3795, acc=88.04%

GRU_h128 Train Epoch 4 [0/60000] Loss: 0.171297
GRU_h128 Train Epoch 4 [19200/60000] Loss: 0.340903
GRU_h128 Train Epoch 4 [38400/60000] Loss: 0.393626
GRU_h128 Train Epoch 4 [57600/60000] Loss: 0.377239
GRU_h128 epoch 4: train_loss=0.3485, train_acc=0.8924
GRU_h128 test epoch 4: loss=0.4124, acc=86.54%

GRU_h128 Train Epoch 5 [0/60000] Loss: 0.485844
GRU_h128 Train Epoch 5 [19200/60000] Loss: 0.298269
GRU_h128 Train Epoch 5 [38400/60000] Loss: 0.177072
GRU_h128 Train Epoch 5 [57600/60000] Loss: 0.458803
GRU_h128 epoch 5: train_loss=0.3563, train_acc=0.8890
GRU_h128 test epoch 5: loss=0.3500, acc=89.04%


Final test accuracies:
RNN_h64 0.4507
RNN_h128 0.4641
RNN_h256 0.0958
LSTM_h128 0.9274
GRU_h128 0.8904
```

In [8]:
```
# Per-epoch comparison for the plain RNN hidden-size sweep
rnn_tags = ['RNN_h64', 'RNN_h128', 'RNN_h256']
```

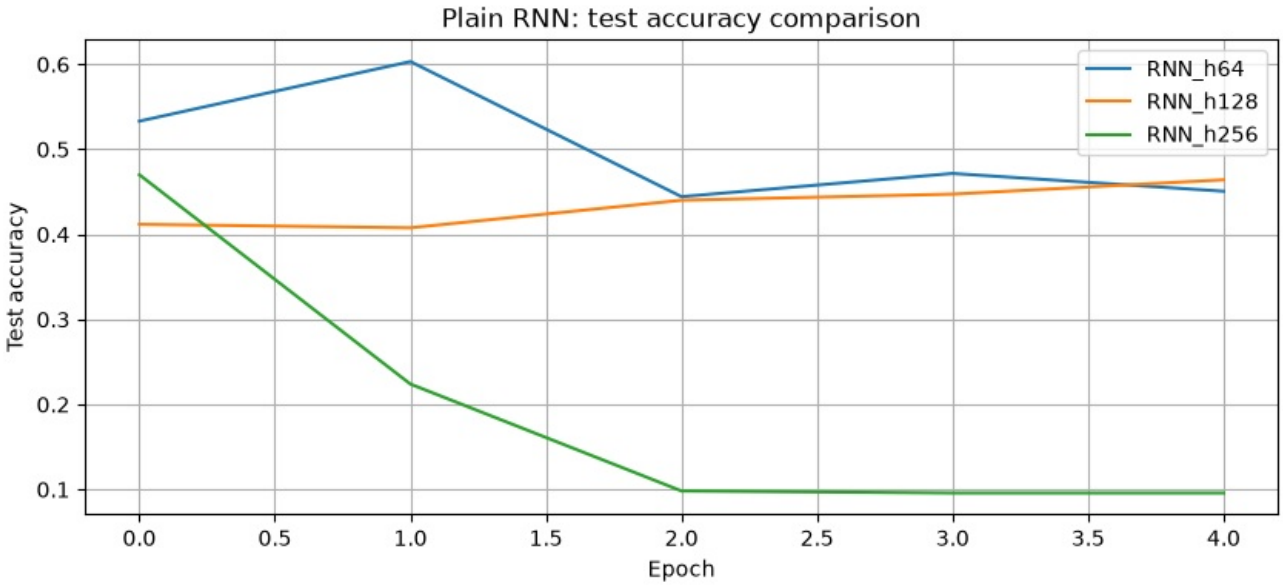
```
plt.figure(figsize=(10, 4))
for tag in rnn_tags:
    plt.plot(all_hist[tag]['test_acc'], label=tag)
plt.title('Plain RNN: test accuracy comparison')
plt.xlabel('Epoch'); plt.ylabel('Test accuracy')
plt.legend(); plt.grid(True); plt.show()

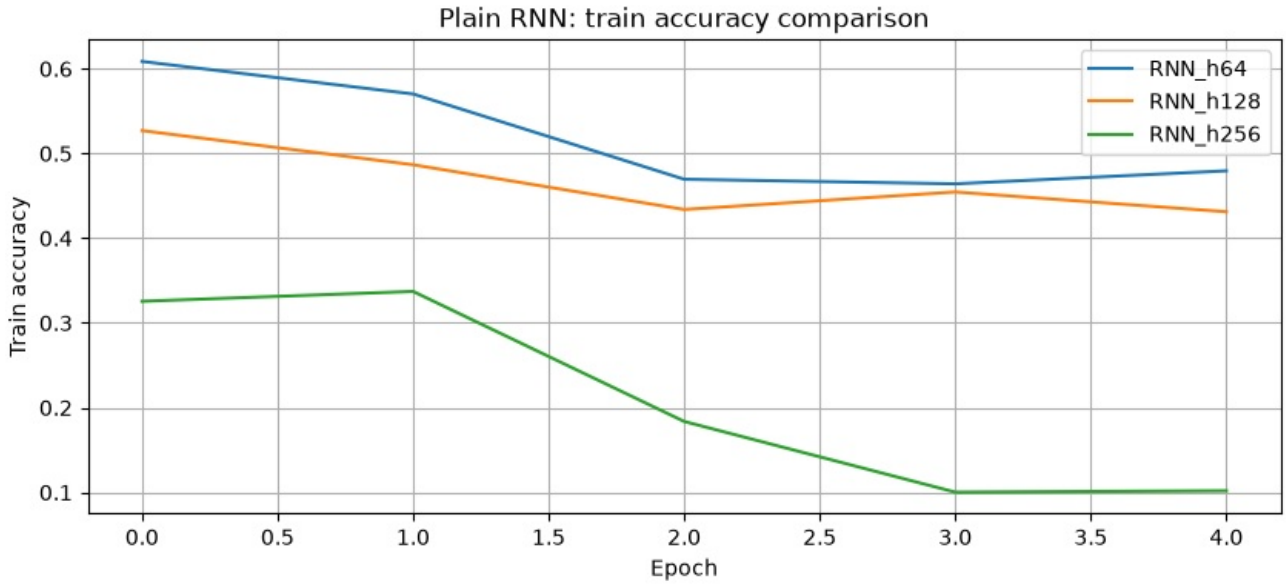
plt.figure(figsize=(10, 4))
for tag in rnn_tags:
    plt.plot(all_hist[tag]['train_acc'], label=tag)
plt.title('Plain RNN: train accuracy comparison')
plt.xlabel('Epoch'); plt.ylabel('Train accuracy')
plt.legend(); plt.grid(True); plt.show()

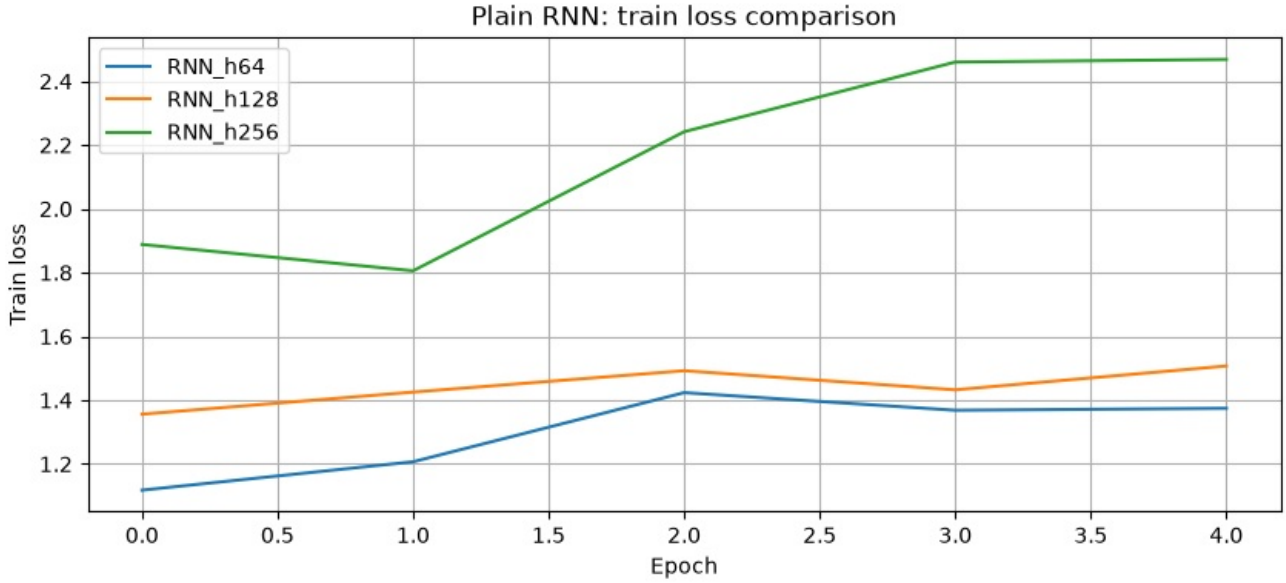
plt.figure(figsize=(10, 4))
for tag in rnn_tags:
    plt.plot(all_hist[tag]['train_loss'], label=tag)
plt.title('Plain RNN: train loss comparison')
plt.xlabel('Epoch'); plt.ylabel('Train loss')
plt.legend(); plt.grid(True); plt.show()

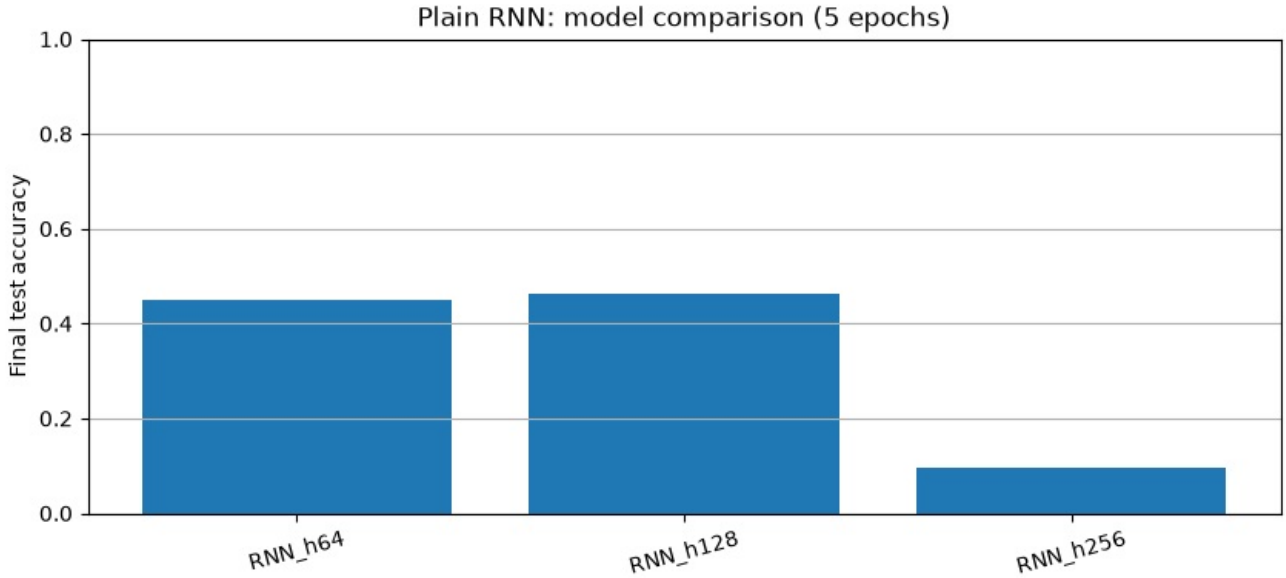
# Final test accuracy bar chart for the plain RNN sweep
plt.figure(figsize=(10, 4))
final_accs = [all_hist[t]['test_acc'][-1] for t in rnn_tags]
plt.bar(rnn_tags, final_accs)
plt.ylabel('Final test accuracy')
plt.title('Plain RNN: model comparison (5 epochs)')
plt.ylim(0, 1); plt.grid(axis='y'); plt.xticks(rotation=15); plt.show()

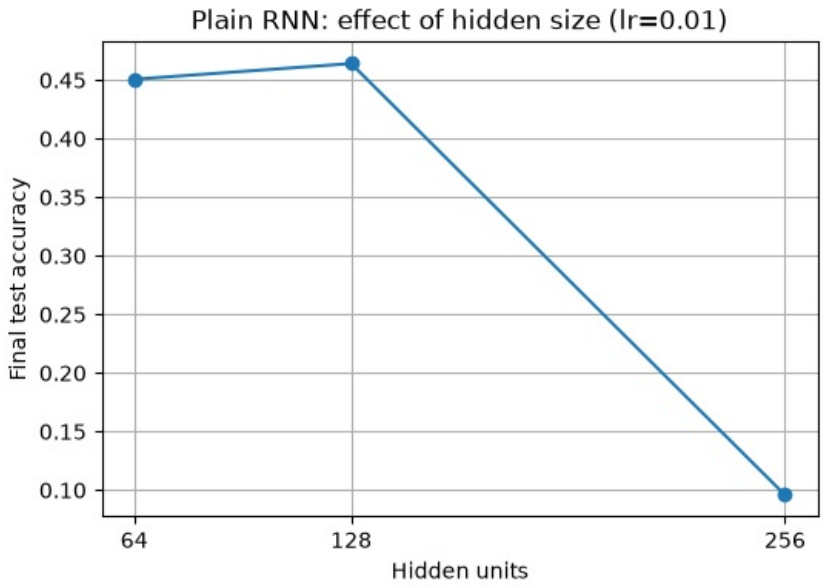
# Hidden-size effect for the plain RNN
sizes = [64, 128, 256]
accs = [all_hist[f'RNN_h{s}']['test_acc'][-1] for s in sizes]
plt.figure(figsize=(6, 4))
plt.plot(sizes, accs, marker='o')
plt.xlabel('Hidden units'); plt.ylabel('Final test accuracy')
plt.title('Plain RNN: effect of hidden size (lr=0.01)')
plt.grid(True); plt.xticks(sizes); plt.show()
```




Plain RNN: train accuracy comparison
RNN_h64
RNN_h128
RNN_h256
Train accuracy
0.6
0.5
0.4
0.3
0.2
0.1
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Epoch


Plain RNN: train loss comparison
RNN_h64
RNN_h128
RNN_h256
Train loss
2.4
2.2
2.0
1.8
1.6
1.4
1.2
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Epoch





3(b) RNN vs. LSTM vs. GRU — observations

All results below use Adam, lr = 0.01, and 5 epochs. To avoid mixing separate runs, the hidden-size table uses Cell 7 for the plain RNN and the dedicated sweeps in Cell 10 (LSTM) and Cell 11 (GRU). Cell 7 also contains exploratory LSTM_h128 and GRU_h128 runs that reached **92.74%** and **89.04%** respectively; those are different runs and are **not** used in the hidden-size table. The authoritative LSTM_h128 result is **94.26%** from Cell 10, and the authoritative GRU_h128 result is **92.04%** from Cell 11.

Plain RNN hidden-size sweep (Cell 7).

- h = 64: final train/test loss **1.3745 / 1.4455**, test accuracy **45.07%**
- h = 128: final train/test loss **1.5071 / 1.4192**, test accuracy **46.41%**
- h = 256: final train/test loss **2.4683 / 2.4925**, test accuracy **9.58%**

The plain RNN gets worse as width increases at lr = 0.01. The h = 256 run essentially collapses to chance. This is an optimisation / gradient-stability problem, not a capacity limit: the same h = 256 RNN trained at lr = 1e-3 reaches **95.67%** by epoch 5 in the optional Cell 9 run.

LSTM hidden-size sweep (Cell 10).

- h = 64: final train/test loss **0.1452 / 0.1479**, test accuracy **95.63%**
- h = 128: final train/test loss **0.1752 / 0.1931**, test accuracy **94.26%**
- h = 256: final train/test loss **0.3534 / 0.2233**, test accuracy **93.26%**

All three LSTM sizes are above 93%. Accuracy decreases monotonically as width increases in this run: **95.63% → 94.26% → 93.26%**. Loss is also lowest at h = 64: both final train loss and final test loss are smallest at h = 64.

GRU hidden-size sweep (Cell 11).

- h = 64: final train/test loss **0.2228 / 0.2120**, test accuracy **93.77%**
- h = 128: final train/test loss **0.2955 / 0.2566**, test accuracy **92.04%**
- h = 256: final train/test loss **0.2854 / 0.3507**, test accuracy **89.22%**

The GRU is less sensitive to width than the plain RNN, but in this rerun its accuracy also decreases monotonically with width: **93.77% → 92.04% → 89.22%**. The spread is about **4.55 percentage points**. h = 64 gives the best test accuracy and the lowest test loss; h = 256 gives the worst test accuracy and the highest test loss.

Comparison at h = 128 using the dedicated sweeps. RNN h = 128: **46.41%**; LSTM h = 128: **94.26%**; GRU h = 128: **92.04%**. Both gated models clearly beat the plain RNN: the LSTM is **+47.85 pp** above the RNN and the GRU is **+45.63 pp** above the RNN. The LSTM is slightly ahead of the GRU by **2.22 pp** at this width.

Are LSTM / GRU better? Yes. The plain RNN is unstable at lr = 0.01, while both gated variants converge to >89% test accuracy. Best overall results in the sweeps: LSTM h = 64 → **95.63%**, LSTM h = 128 → **94.26%**, GRU h = 64 → **93.77%**, LSTM h = 256 → **93.26%**, GRU h = 128 → **92.04%**, GRU h = 256 → **89.22%**.

GRU / LSTM loss plots. The two dedicated figures are produced in Cell 12:

1. LSTM vs. GRU train/test loss at h = 128.
2. RNN vs. LSTM vs. GRU train/test loss at h = 128.

These are the figures that satisfy the “plots for LSTM/GRU loss” checklist item.

In [9]:
```
# Optional: RNN with 256 hidden units at a stable learning rate
hist_h256_lr3 = run_mnist_experiment(rnn_type='RNN', hidden_size=256,
                                     n_epochs=5, lr=1e-3)
all_hist['RNN_h256_lr1e-3'] = hist_h256_lr3
print('RNN_h256 lr=1e-3 final test acc:', hist_h256_lr3['test_acc'][-1])

plt.figure(figsize=(6, 4))
plt.plot(hist_h256_lr3['test_acc'], marker='o')
plt.title('RNN h=256, lr=1e-3')
plt.xlabel('Epoch'); plt.ylabel('Test accuracy'); plt.grid(True)
plt.show()
```

```
RNN_h256 Train Epoch 1 [0/60000] Loss: 2.309808
RNN_h256 Train Epoch 1 [19200/60000] Loss: 0.592946
RNN_h256 Train Epoch 1 [38400/60000] Loss: 0.324122
RNN_h256 Train Epoch 1 [57600/60000] Loss: 0.229599
RNN_h256 epoch 1: train_loss=0.5318, train_acc=0.8317
RNN_h256 test epoch 1: loss=0.2588, acc=92.58%

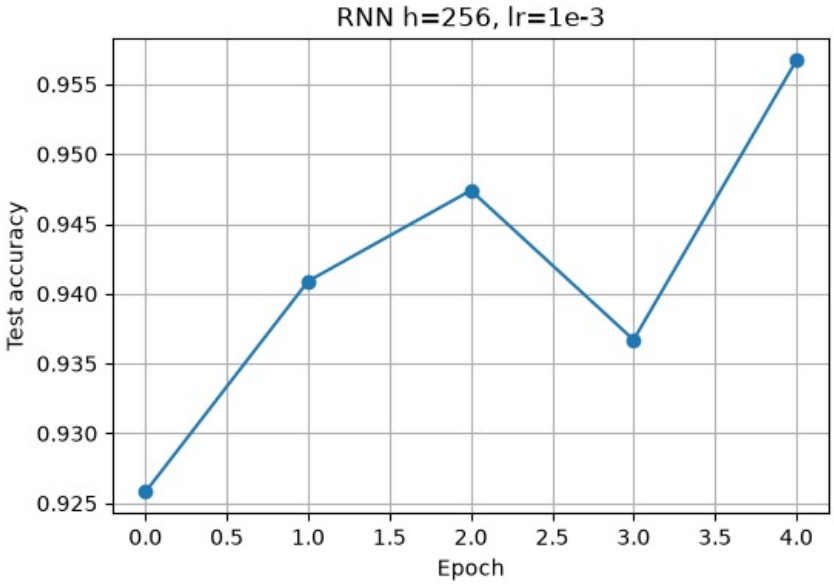
RNN_h256 Train Epoch 2 [0/60000] Loss: 0.128643
RNN_h256 Train Epoch 2 [19200/60000] Loss: 0.243712
RNN_h256 Train Epoch 2 [38400/60000] Loss: 0.167798
RNN_h256 Train Epoch 2 [57600/60000] Loss: 0.198104
RNN_h256 epoch 2: train_loss=0.2503, train_acc=0.9287
RNN_h256 test epoch 2: loss=0.2179, acc=94.09%

RNN_h256 Train Epoch 3 [0/60000] Loss: 0.284441
RNN_h256 Train Epoch 3 [19200/60000] Loss: 0.086565
RNN_h256 Train Epoch 3 [38400/60000] Loss: 0.298105
RNN_h256 Train Epoch 3 [57600/60000] Loss: 0.269094
RNN_h256 epoch 3: train_loss=0.1912, train_acc=0.9454
RNN_h256 test epoch 3: loss=0.1829, acc=94.74%

RNN_h256 Train Epoch 4 [0/60000] Loss: 0.245873
RNN_h256 Train Epoch 4 [19200/60000] Loss: 0.122423
RNN_h256 Train Epoch 4 [38400/60000] Loss: 0.112976
RNN_h256 Train Epoch 4 [57600/60000] Loss: 0.086518
RNN_h256 epoch 4: train_loss=0.1691, train_acc=0.9518
RNN_h256 test epoch 4: loss=0.2249, acc=93.67%

RNN_h256 Train Epoch 5 [0/60000] Loss: 0.129503
RNN_h256 Train Epoch 5 [19200/60000] Loss: 0.084774
RNN_h256 Train Epoch 5 [38400/60000] Loss: 0.088877
RNN_h256 Train Epoch 5 [57600/60000] Loss: 0.096574
RNN_h256 epoch 5: train_loss=0.1570, train_acc=0.9558
RNN_h256 test epoch 5: loss=0.1701, acc=95.67%

RNN_h256 lr=1e-3 final test acc: 0.9567
```



In [10]:

```python
# ===== 3(b) hidden-size sweep for the LSTM =====
lstm_sweep = {}
for h in [64, 128, 256]:
    tag = f'LSTM_h{h}'
    print(f"\n===== Running {tag} =====")
    lstm_sweep[h] = run_mnist_experiment(
        rnn_type='LSTM', hidden_size=h, n_epochs=5, lr=0.01
    )

print("\nLSTM final test accuracy by hidden size:")
for h, hist in lstm_sweep.items():
    print(f"  h={h:>3d}: {hist['test_acc'][-1]:.4f}")

# Plot: test accuracy per epoch for each hidden size
plt.figure(figsize=(7, 4))
for h, hist in lstm_sweep.items():
    plt.plot(hist['test_acc'], marker='o', label=f'LSTM h={h}')
plt.title('LSTM: effect of hidden size on test accuracy (lr=0.01, 5 epochs)')
plt.xlabel('Epoch'); plt.ylabel('Test accuracy')
plt.legend(); plt.grid(True); plt.show()

# Plot: train + test loss per epoch for each hidden size
plt.figure(figsize=(8, 4))
```

```python
for h, hist in lstm_sweep.items():
    plt.plot(hist['train_loss'], marker='o', label=f'LSTM h={h} train')
    plt.plot(hist['test_loss'],  marker='s', linestyle='--', label=f'LSTM h={h} test')
plt.title('LSTM: effect of hidden size on train & test loss (lr=0.01, 5 epochs)')
plt.xlabel('Epoch'); plt.ylabel('Loss')
plt.legend(fontsize=8); plt.grid(True); plt.show()
```

```
===== Running LSTM_h64 =====
LSTM_h64 Train Epoch 1 [0/60000] Loss: 2.289778
LSTM_h64 Train Epoch 1 [19200/60000] Loss: 0.369464
LSTM_h64 Train Epoch 1 [38400/60000] Loss: 0.141114
LSTM_h64 Train Epoch 1 [57600/60000] Loss: 0.428965
LSTM_h64 epoch 1: train_loss=0.3185, train_acc=0.8985
LSTM_h64 test epoch 1: loss=0.1708, acc=94.96%

LSTM_h64 Train Epoch 2 [0/60000] Loss: 0.062899
LSTM_h64 Train Epoch 2 [19200/60000] Loss: 0.406897
LSTM_h64 Train Epoch 2 [38400/60000] Loss: 0.445092
LSTM_h64 Train Epoch 2 [57600/60000] Loss: 0.146248
LSTM_h64 epoch 2: train_loss=0.1434, train_acc=0.9576
LSTM_h64 test epoch 2: loss=0.1389, acc=95.89%

LSTM_h64 Train Epoch 3 [0/60000] Loss: 0.077696
LSTM_h64 Train Epoch 3 [19200/60000] Loss: 0.150467
LSTM_h64 Train Epoch 3 [38400/60000] Loss: 0.409664
LSTM_h64 Train Epoch 3 [57600/60000] Loss: 0.132320
LSTM_h64 epoch 3: train_loss=0.1273, train_acc=0.9621
LSTM_h64 test epoch 3: loss=0.1232, acc=96.48%

LSTM_h64 Train Epoch 4 [0/60000] Loss: 0.163019
LSTM_h64 Train Epoch 4 [19200/60000] Loss: 0.125497
LSTM_h64 Train Epoch 4 [38400/60000] Loss: 0.059596
LSTM_h64 Train Epoch 4 [57600/60000] Loss: 0.061443
LSTM_h64 epoch 4: train_loss=0.1232, train_acc=0.9635
LSTM_h64 test epoch 4: loss=0.1345, acc=95.81%

LSTM_h64 Train Epoch 5 [0/60000] Loss: 0.206951
LSTM_h64 Train Epoch 5 [19200/60000] Loss: 0.043668
LSTM_h64 Train Epoch 5 [38400/60000] Loss: 0.118457
LSTM_h64 Train Epoch 5 [57600/60000] Loss: 0.233900
LSTM_h64 epoch 5: train_loss=0.1452, train_acc=0.9565
LSTM_h64 test epoch 5: loss=0.1479, acc=95.63%


===== Running LSTM_h128 =====
LSTM_h128 Train Epoch 1 [0/60000] Loss: 2.293481
LSTM_h128 Train Epoch 1 [19200/60000] Loss: 0.333700
LSTM_h128 Train Epoch 1 [38400/60000] Loss: 0.115668
LSTM_h128 Train Epoch 1 [57600/60000] Loss: 0.328575
LSTM_h128 epoch 1: train_loss=0.2869, train_acc=0.9099
LSTM_h128 test epoch 1: loss=0.1593, acc=95.31%

LSTM_h128 Train Epoch 2 [0/60000] Loss: 0.143114
LSTM_h128 Train Epoch 2 [19200/60000] Loss: 0.207370
LSTM_h128 Train Epoch 2 [38400/60000] Loss: 0.103456
LSTM_h128 Train Epoch 2 [57600/60000] Loss: 0.110127
LSTM_h128 epoch 2: train_loss=0.1484, train_acc=0.9555
LSTM_h128 test epoch 2: loss=0.1264, acc=96.37%

LSTM_h128 Train Epoch 3 [0/60000] Loss: 0.067824
LSTM_h128 Train Epoch 3 [19200/60000] Loss: 0.045865
LSTM_h128 Train Epoch 3 [38400/60000] Loss: 0.169501
LSTM_h128 Train Epoch 3 [57600/60000] Loss: 0.058352
LSTM_h128 epoch 3: train_loss=0.1523, train_acc=0.9543
LSTM_h128 test epoch 3: loss=0.1455, acc=95.64%

LSTM_h128 Train Epoch 4 [0/60000] Loss: 0.238991
LSTM_h128 Train Epoch 4 [19200/60000] Loss: 0.141903
LSTM_h128 Train Epoch 4 [38400/60000] Loss: 0.241656
LSTM_h128 Train Epoch 4 [57600/60000] Loss: 0.084343
LSTM_h128 epoch 4: train_loss=0.1625, train_acc=0.9515
LSTM_h128 test epoch 4: loss=0.1398, acc=95.82%

LSTM_h128 Train Epoch 5 [0/60000] Loss: 0.156034
LSTM_h128 Train Epoch 5 [19200/60000] Loss: 0.265382
LSTM_h128 Train Epoch 5 [38400/60000] Loss: 0.241162
LSTM_h128 Train Epoch 5 [57600/60000] Loss: 0.136666
LSTM_h128 epoch 5: train_loss=0.1752, train_acc=0.9475
LSTM_h128 test epoch 5: loss=0.1931, acc=94.26%


===== Running LSTM_h256 =====
LSTM_h256 Train Epoch 1 [0/60000] Loss: 2.309051
LSTM_h256 Train Epoch 1 [19200/60000] Loss: 0.087714
```

```
LSTM_h256 Train Epoch 1 [38400/60000] Loss: 0.082057
LSTM_h256 Train Epoch 1 [57600/60000] Loss: 0.182595
LSTM_h256 epoch 1: train_loss=0.2839, train_acc=0.9109
LSTM_h256 test epoch 1: loss=0.1709, acc=94.69%

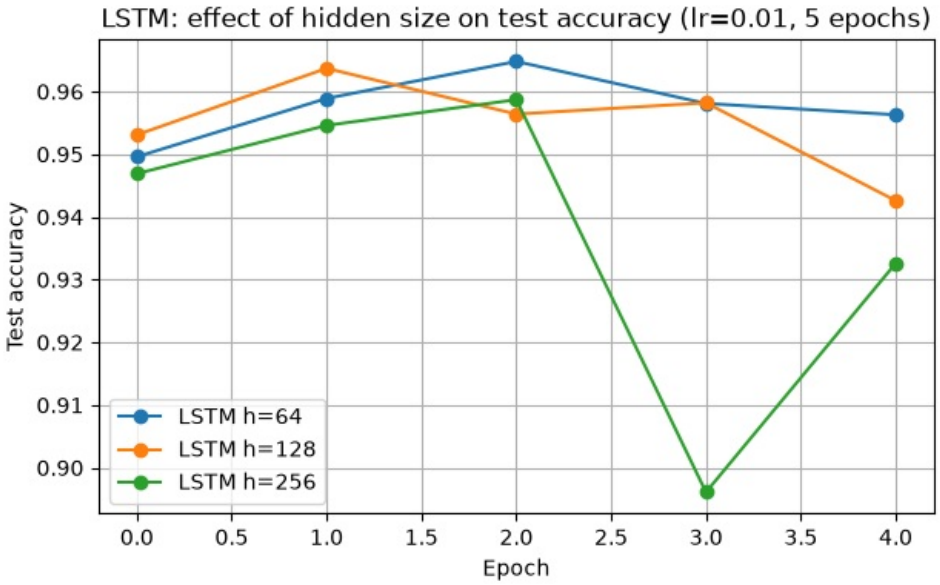
LSTM_h256 Train Epoch 2 [0/60000] Loss: 0.242118
LSTM_h256 Train Epoch 2 [19200/60000] Loss: 0.103274
LSTM_h256 Train Epoch 2 [38400/60000] Loss: 0.347634
LSTM_h256 Train Epoch 2 [57600/60000] Loss: 0.198817
LSTM_h256 epoch 2: train_loss=0.1509, train_acc=0.9552
LSTM_h256 test epoch 2: loss=0.1483, acc=95.46%

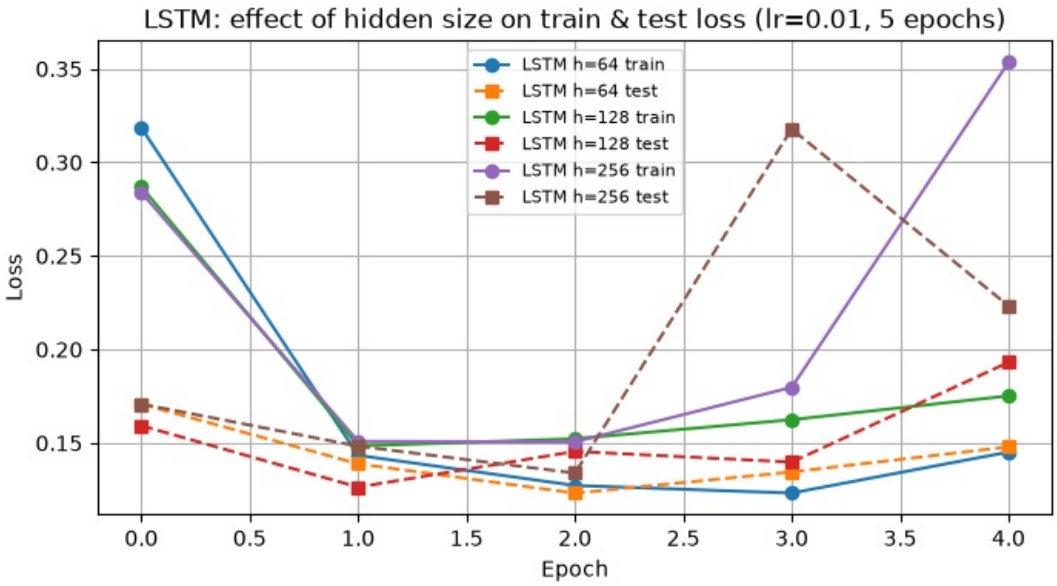
LSTM_h256 Train Epoch 3 [0/60000] Loss: 0.342878
LSTM_h256 Train Epoch 3 [19200/60000] Loss: 0.095470
LSTM_h256 Train Epoch 3 [38400/60000] Loss: 0.241858
LSTM_h256 Train Epoch 3 [57600/60000] Loss: 0.347805
LSTM_h256 epoch 3: train_loss=0.1506, train_acc=0.9552
LSTM_h256 test epoch 3: loss=0.1339, acc=95.87%

LSTM_h256 Train Epoch 4 [0/60000] Loss: 0.173765
LSTM_h256 Train Epoch 4 [19200/60000] Loss: 0.494963
LSTM_h256 Train Epoch 4 [38400/60000] Loss: 0.076067
LSTM_h256 Train Epoch 4 [57600/60000] Loss: 0.299886
LSTM_h256 epoch 4: train_loss=0.1797, train_acc=0.9469
LSTM_h256 test epoch 4: loss=0.3178, acc=89.61%

LSTM_h256 Train Epoch 5 [0/60000] Loss: 0.366686
LSTM_h256 Train Epoch 5 [19200/60000] Loss: 0.390585
LSTM_h256 Train Epoch 5 [38400/60000] Loss: 0.197494
LSTM_h256 Train Epoch 5 [57600/60000] Loss: 0.263999
LSTM_h256 epoch 5: train_loss=0.3534, train_acc=0.8908
LSTM_h256 test epoch 5: loss=0.2233, acc=93.26%


LSTM final test accuracy by hidden size:
  h= 64: 0.9563
  h=128: 0.9426
  h=256: 0.9326
```





In [11]:

```python
# ===== 3(b) hidden-size sweep for the GRU =====
gru_sweep = {}
for h in [64, 128, 256]:
    tag = f'GRU_h{h}'
    print(f"\n===== Running {tag} =====")
    gru_sweep[h] = run_mnist_experiment(
        rnn_type='GRU', hidden_size=h, n_epochs=5, lr=0.01
    )

print("\nGRU final test accuracy by hidden size:")
for h, hist in gru_sweep.items():
    print(f"  h={h:>3d}: {hist['test_acc'][-1]:.4f}")

# Test accuracy per epoch
plt.figure(figsize=(7, 4))
for h, hist in gru_sweep.items():
    plt.plot(hist['test_acc'], marker='o', label=f'GRU h={h}')
plt.title('GRU: effect of hidden size on test accuracy (lr=0.01, 5 epochs)')
plt.xlabel('Epoch'); plt.ylabel('Test accuracy')
plt.legend(); plt.grid(True); plt.show()

# Train + test loss per epoch
plt.figure(figsize=(8, 4))
for h, hist in gru_sweep.items():
    plt.plot(hist['train_loss'], marker='o', label=f'GRU h={h} train')
    plt.plot(hist['test_loss'],  marker='s', linestyle='--', label=f'GRU h={h} test')
plt.title('GRU: effect of hidden size on train & test loss (lr=0.01, 5 epochs)')
plt.xlabel('Epoch'); plt.ylabel('Loss')
plt.legend(fontsize=8); plt.grid(True); plt.show()
```

```
===== Running GRU_h64 =====
GRU_h64 Train Epoch 1 [0/60000] Loss: 2.325495
GRU_h64 Train Epoch 1 [19200/60000] Loss: 0.188015
GRU_h64 Train Epoch 1 [38400/60000] Loss: 0.172240
GRU_h64 Train Epoch 1 [57600/60000] Loss: 0.182099
GRU_h64 epoch 1: train_loss=0.2674, train_acc=0.9137
GRU_h64 test epoch 1: loss=0.1315, acc=96.11%
```

```
GRU_h64 Train Epoch 2 [0/60000] Loss: 0.068794
GRU_h64 Train Epoch 2 [19200/60000] Loss: 0.156768
GRU_h64 Train Epoch 2 [38400/60000] Loss: 0.191756
GRU_h64 Train Epoch 2 [57600/60000] Loss: 0.286521
GRU_h64 epoch 2: train_loss=0.1417, train_acc=0.9572
GRU_h64 test epoch 2: loss=0.1537, acc=95.69%

GRU_h64 Train Epoch 3 [0/60000] Loss: 0.090489
GRU_h64 Train Epoch 3 [19200/60000] Loss: 0.084639
GRU_h64 Train Epoch 3 [38400/60000] Loss: 0.212222
GRU_h64 Train Epoch 3 [57600/60000] Loss: 0.094260
GRU_h64 epoch 3: train_loss=0.1649, train_acc=0.9512
GRU_h64 test epoch 3: loss=0.1719, acc=94.84%

GRU_h64 Train Epoch 4 [0/60000] Loss: 0.209890
GRU_h64 Train Epoch 4 [19200/60000] Loss: 0.096054
GRU_h64 Train Epoch 4 [38400/60000] Loss: 0.273683
GRU_h64 Train Epoch 4 [57600/60000] Loss: 0.305232
GRU_h64 epoch 4: train_loss=0.2064, train_acc=0.9368
GRU_h64 test epoch 4: loss=0.1962, acc=93.96%

GRU_h64 Train Epoch 5 [0/60000] Loss: 0.153113
GRU_h64 Train Epoch 5 [19200/60000] Loss: 0.310576
GRU_h64 Train Epoch 5 [38400/60000] Loss: 0.178638
GRU_h64 Train Epoch 5 [57600/60000] Loss: 0.045281
GRU_h64 epoch 5: train_loss=0.2228, train_acc=0.9329
GRU_h64 test epoch 5: loss=0.2120, acc=93.77%


===== Running GRU_h128 =====
GRU_h128 Train Epoch 1 [0/60000] Loss: 2.310469
GRU_h128 Train Epoch 1 [19200/60000] Loss: 0.171316
GRU_h128 Train Epoch 1 [38400/60000] Loss: 0.128863
GRU_h128 Train Epoch 1 [57600/60000] Loss: 0.277039
GRU_h128 epoch 1: train_loss=0.2406, train_acc=0.9241
GRU_h128 test epoch 1: loss=0.1869, acc=94.59%

GRU_h128 Train Epoch 2 [0/60000] Loss: 0.180119
GRU_h128 Train Epoch 2 [19200/60000] Loss: 0.200710
GRU_h128 Train Epoch 2 [38400/60000] Loss: 0.261142
GRU_h128 Train Epoch 2 [57600/60000] Loss: 0.333403
GRU_h128 epoch 2: train_loss=0.2431, train_acc=0.9272
GRU_h128 test epoch 2: loss=0.2694, acc=91.76%

GRU_h128 Train Epoch 3 [0/60000] Loss: 0.086297
GRU_h128 Train Epoch 3 [19200/60000] Loss: 0.176209
GRU_h128 Train Epoch 3 [38400/60000] Loss: 0.777222
GRU_h128 Train Epoch 3 [57600/60000] Loss: 0.388230
GRU_h128 epoch 3: train_loss=0.4716, train_acc=0.8532
GRU_h128 test epoch 3: loss=0.7562, acc=75.11%

GRU_h128 Train Epoch 4 [0/60000] Loss: 0.823378
GRU_h128 Train Epoch 4 [19200/60000] Loss: 0.248131
GRU_h128 Train Epoch 4 [38400/60000] Loss: 0.718086
GRU_h128 Train Epoch 4 [57600/60000] Loss: 0.318611
GRU_h128 epoch 4: train_loss=0.4134, train_acc=0.8677
GRU_h128 test epoch 4: loss=0.3365, acc=88.67%

GRU_h128 Train Epoch 5 [0/60000] Loss: 0.619288
GRU_h128 Train Epoch 5 [19200/60000] Loss: 0.529852
GRU_h128 Train Epoch 5 [38400/60000] Loss: 0.582579
GRU_h128 Train Epoch 5 [57600/60000] Loss: 0.414443
GRU_h128 epoch 5: train_loss=0.2955, train_acc=0.9060
GRU_h128 test epoch 5: loss=0.2566, acc=92.04%


===== Running GRU_h256 =====
GRU_h256 Train Epoch 1 [0/60000] Loss: 2.307720
GRU_h256 Train Epoch 1 [19200/60000] Loss: 0.084447
GRU_h256 Train Epoch 1 [38400/60000] Loss: 0.248991
GRU_h256 Train Epoch 1 [57600/60000] Loss: 0.911113
GRU_h256 epoch 1: train_loss=0.3992, train_acc=0.8749
GRU_h256 test epoch 1: loss=0.4789, acc=85.11%

GRU_h256 Train Epoch 2 [0/60000] Loss: 0.478035
GRU_h256 Train Epoch 2 [19200/60000] Loss: 0.375116
GRU_h256 Train Epoch 2 [38400/60000] Loss: 0.653319
GRU_h256 Train Epoch 2 [57600/60000] Loss: 0.341565
GRU_h256 epoch 2: train_loss=0.5398, train_acc=0.8310
GRU_h256 test epoch 2: loss=0.7477, acc=75.67%

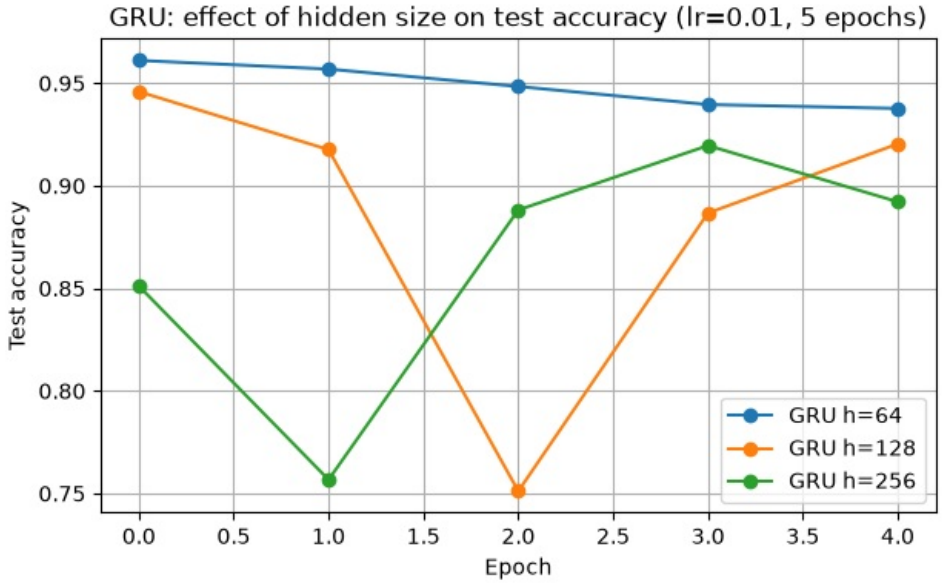
GRU_h256 Train Epoch 3 [0/60000] Loss: 0.716151
```

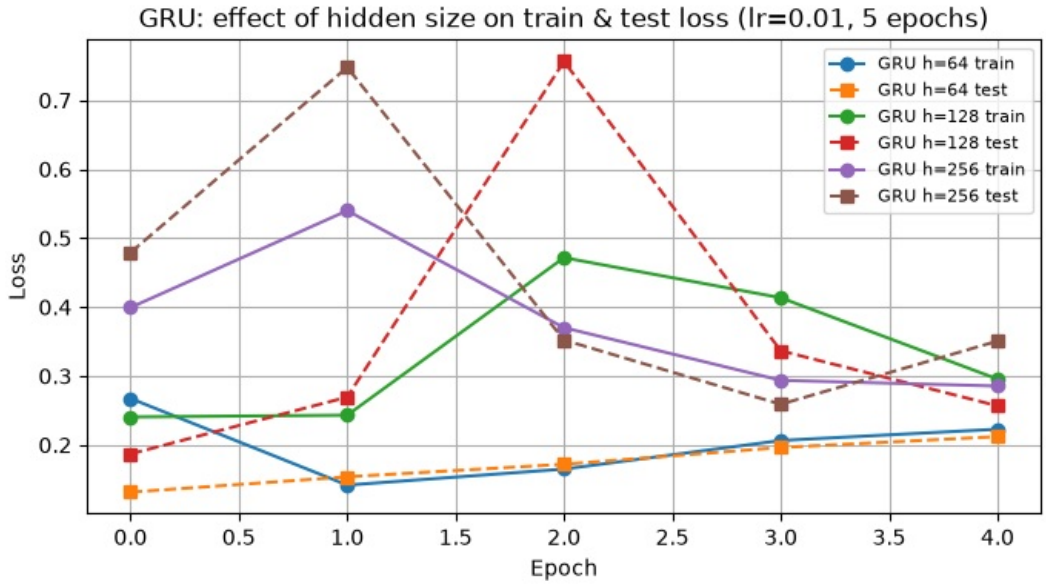
```
GRU_h256 Train Epoch 3 [19200/60000] Loss: 0.153224
GRU_h256 Train Epoch 3 [38400/60000] Loss: 0.547471
GRU_h256 Train Epoch 3 [57600/60000] Loss: 0.233554
GRU_h256 epoch 3: train_loss=0.3700, train_acc=0.8850
GRU_h256 test epoch 3: loss=0.3521, acc=88.82%

GRU_h256 Train Epoch 4 [0/60000] Loss: 0.244748
GRU_h256 Train Epoch 4 [19200/60000] Loss: 0.417898
GRU_h256 Train Epoch 4 [38400/60000] Loss: 0.074843
GRU_h256 Train Epoch 4 [57600/60000] Loss: 0.462593
GRU_h256 epoch 4: train_loss=0.2937, train_acc=0.9096
GRU_h256 test epoch 4: loss=0.2589, acc=91.96%

GRU_h256 Train Epoch 5 [0/60000] Loss: 0.219943
GRU_h256 Train Epoch 5 [19200/60000] Loss: 0.356308
GRU_h256 Train Epoch 5 [38400/60000] Loss: 0.233038
GRU_h256 Train Epoch 5 [57600/60000] Loss: 0.226419
GRU_h256 epoch 5: train_loss=0.2854, train_acc=0.9152
GRU_h256 test epoch 5: loss=0.3507, acc=89.22%


GRU final test accuracy by hidden size:
  h= 64: 0.9377
  h=128: 0.9204
  h=256: 0.8922
```



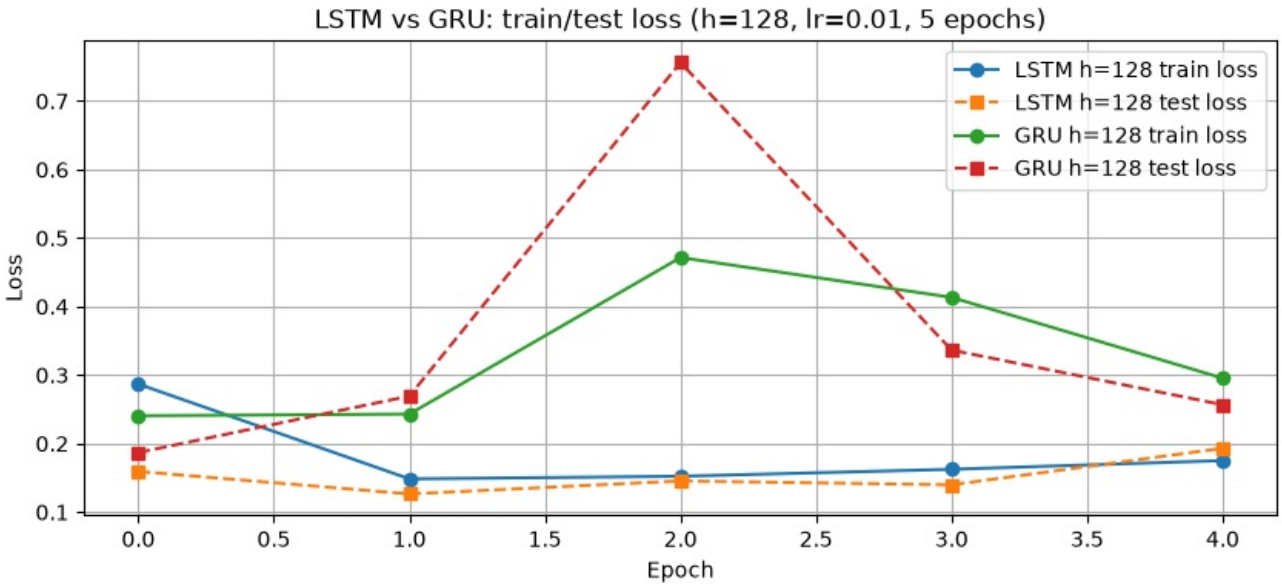


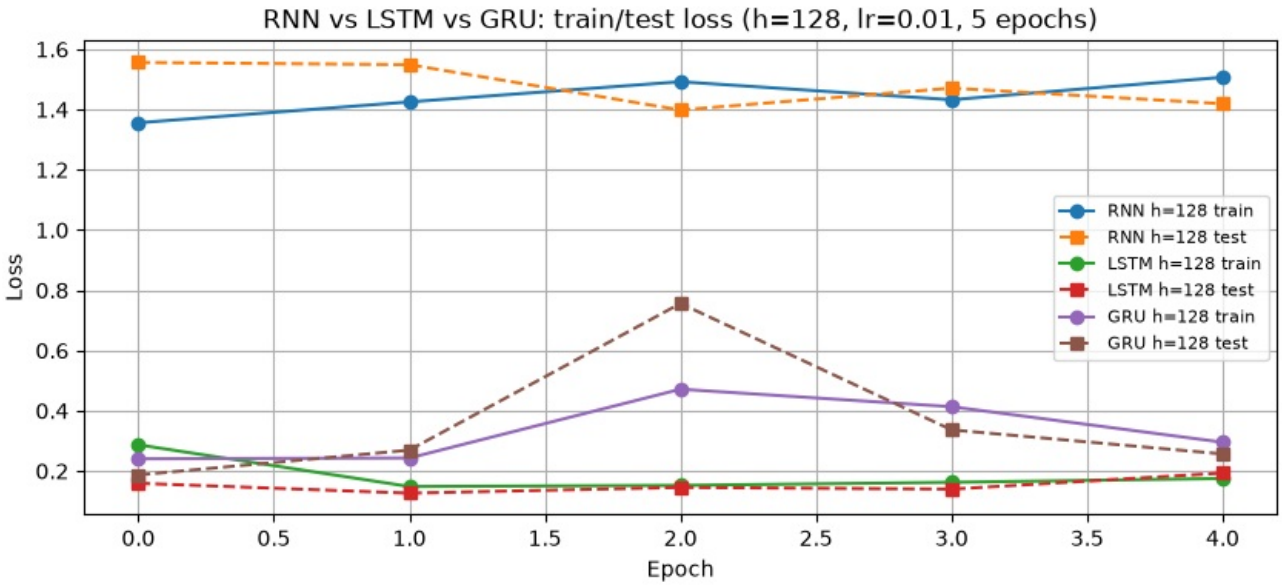
In [12]:
```python
# ===== Dedicated LSTM vs GRU loss comparison (train + test) =====
plt.figure(figsize=(10, 4))
for name, hist in [('LSTM h=128', lstm_sweep[128]), ('GRU h=128', gru_sweep[128])]:
    plt.plot(hist['train_loss'], marker='o', label=f'{name} train loss')
    plt.plot(hist['test_loss'],  marker='s', linestyle='--', label=f'{name} test loss')
plt.title('LSTM vs GRU: train/test loss (h=128, lr=0.01, 5 epochs)')
plt.xlabel('Epoch'); plt.ylabel('Loss')
plt.legend(); plt.grid(True); plt.show()

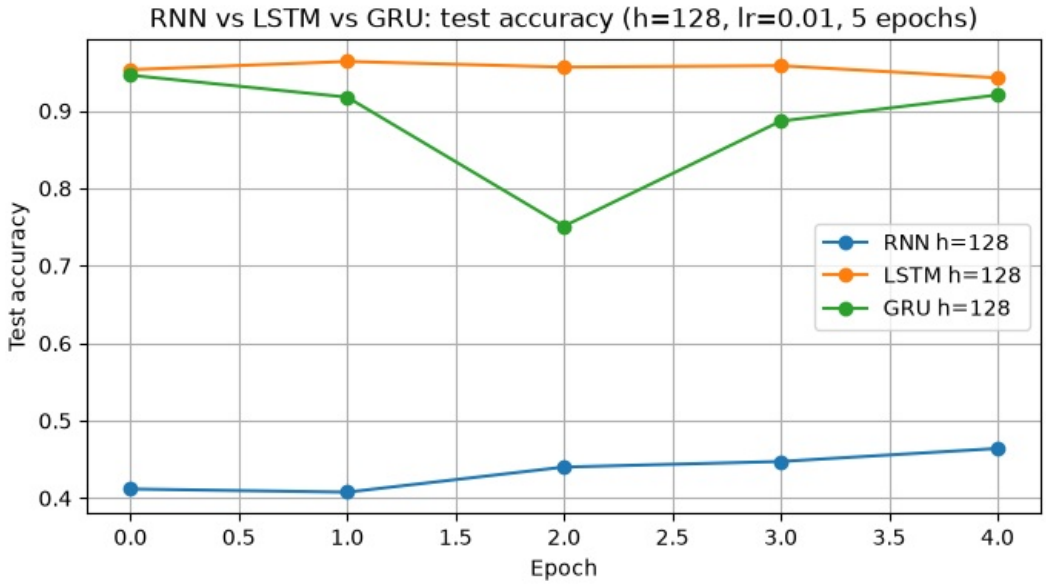
# ===== RNN vs LSTM vs GRU train/test loss (h=128) =====
plt.figure(figsize=(10, 4))
```

```
series = [
    ('RNN h=128',  all_hist['RNN_h128']),
    ('LSTM h=128', lstm_sweep[128]),
    ('GRU h=128',  gru_sweep[128]),
]
for name, hist in series:
    plt.plot(hist['train_loss'], marker='o', label=f'{name} train')
    plt.plot(hist['test_loss'],  marker='s', linestyle='--', label=f'{name} test')
plt.title('RNN vs LSTM vs GRU: train/test loss (h=128, lr=0.01, 5 epochs)')
plt.xlabel('Epoch'); plt.ylabel('Loss')
plt.legend(fontsize=8); plt.grid(True); plt.show()

# ===== RNN vs LSTM vs GRU test accuracy (h=128) =====
plt.figure(figsize=(8, 4))
for name, hist in series:
    plt.plot(hist['test_acc'], marker='o', label=name)
plt.title('RNN vs LSTM vs GRU: test accuracy (h=128, lr=0.01, 5 epochs)')
plt.xlabel('Epoch'); plt.ylabel('Test accuracy')
plt.legend(); plt.grid(True); plt.show()
```







In [13]:
```
import pandas as pd

rows = []
for h in (64, 128, 256):
    for name, hist in [
        ('RNN',  all_hist[f'RNN_h{h}']),
        ('LSTM', lstm_sweep[h]),
        ('GRU',  gru_sweep[h]),
    ]:
        rows.append({
            'Model': name,
            'Hidden': h,
            'Final train loss': hist['train_loss'][-1],
            'Final test loss': hist['test_loss'][-1],
            'Final test acc %': 100 * hist['test_acc'][-1],
        })

summary = pd.DataFrame(rows)
display(summary.round(4))
```

	Model	Hidden	Final train loss	Final test loss	Final test acc %
0	RNN	64	1.3745	1.4455	45.07
1	LSTM	64	0.1452	0.1479	95.63
2	GRU	64	0.2228	0.2120	93.77
3	RNN	128	1.5071	1.4192	46.41
4	LSTM	128	0.1752	0.1931	94.26
5	GRU	128	0.2955	0.2566	92.04
6	RNN	256	2.4683	2.4925	9.58
7	LSTM	256	0.3534	0.2233	93.26
8	GRU	256	0.2854	0.3507	89.22

In [14]:
```
writer.close()
```

3(c) Comparison with the CNN from Assignment 1

CNN baseline. In Assignment 1, the best CNN reached **99.30%** test accuracy on MNIST (SGD + Momentum run). The LeNet5 result on CIFAR-10 in Problem 1 is not a fair MNIST comparison because it uses a different dataset; the relevant baseline is the Assignment 1 MNIST CNN.

RNN / gated RNN results on MNIST. Using the dedicated Part 3 sweeps:

- Plain RNN h = 128: **46.41%** at 5 epochs.
- LSTM h = 128: **94.26%** at 5 epochs.
- GRU h = 128: **92.04%** at 5 epochs.
- Best LSTM in the sweep: h = 64, **95.63%**.
- Best GRU in the sweep: h = 64, **93.77%**.

The plain RNN is also unstable: the 10-epoch baseline peaks at **53.90%** and ends at **30.06%**, with large epoch-to-epoch swings in train loss and test accuracy.

Quantitative gap. Using the Assignment 1 CNN accuracy **99.30%**:

- CNN vs. plain RNN h = 128: **99.30 − 46.41 = 52.89 pp**.
- CNN vs. LSTM h = 128: **99.30 − 94.26 = 5.04 pp**.
- CNN vs. GRU h = 128: **99.30 − 92.04 = 7.26 pp**.
- CNN vs. best LSTM h = 64: **99.30 − 95.63 = 3.67 pp**.
- CNN vs. best GRU h = 64: **99.30 − 93.77 = 5.53 pp**.

The CNN outperformed the plain RNN by about **52.89 percentage points**, and still outperformed the best gated RNN by roughly **3.67–5.53 percentage points**. These are percentage-point gaps, not percent gaps.

Why the CNN wins. A CNN encodes 2-D locality, weight sharing, and translation equivariance directly into the architecture. A row-wise RNN treats each 28×28 image as 28 sequential 28-vector inputs, so vertical spatial structure must be learned through 28 recurrent updates. The plain RNN also suffers from vanishing / exploding gradients, which is exactly what the h = 256 collapse to **9.58%** at lr = 0.01 demonstrates. LSTM / GRU gates largely fix the gradient problem and recover most of the accuracy, but the recurrent models are still sequential over time steps and do not have the CNN's 2-D inductive bias.

Similarities. Both are trained end-to-end with cross-entropy, both learn hierarchical features, and both are sensitive to learning rate and input normalisation.

Differences / practical takeaway. CNNs are the right default for static images. RNNs / LSTMs / GRUs are preferable when the data is genuinely sequential, e.g. handwriting strokes, audio, or video frames. Gated RNNs make row-wise MNIST trainable, but even the best gated RNN in this notebook remains below the Assignment 1 CNN.